DAGING · RISET MENDALAM

Dari Kode ke Chip: Cara Compiler Berbicara dengan Hardware

Perjalanan sebuah baris kode dari bahasa pemrograman, compiler, linker, hingga transistor di mikrokontroler, termasuk memory alignment, volatile, cache, DMA, dan studi kasus pembaca serta tag RFID

Dokumen dibuat 12 November 2026. Disusun dari riset berbagai sumber; rujukan tiap klaim tercantum sebagai catatan kaki di akhir dokumen. Data dan angka berlaku per tanggal sumbernya.

Daftar Isi

Pendahuluan

Gambaran Besar

- Tujuh lapisan dari ide ke elektron
- Tiga ide yang menjelaskan hampir semuanya
- Fakta kunci

Sejarah Singkat: Dari Kabel ke Compiler

- Ketika "memprogram" berarti memasang kabel
- FORTRAN: bukti bahwa mesin bisa menulis kode yang cepat
- C: bahasa yang "dekat dengan logam"
- GCC dan LLVM: dua raksasa compiler terbuka

Bahasa Pemrograman sebagai Kontrak

- Mesin abstrak dan aturan "seolah-olah"
- Tiga jenis "ketidakpastian" yang diizinkan standar
- Tipe data: bahasa yang berbicara tentang ukuran dan alamat

Di Dalam Compiler

- Peta besar: front-end, middle-end, back-end
- Tahap 1: dari teks ke pohon (lexing, parsing, analisis semantik)
- Tahap 2: representasi antara dan bentuk SSA
- Tahap 3: optimisasi
- Tahap 4: back-end, dari IR ke instruksi nyata
- Mengapa level optimisasi mengubah perilaku program

Assembler, Linker, dan Kode Startup

- File objek: kode yang belum tahu alamatnya
- Linker: menyusun peta memori
- Saat listrik menyala: urutan reset Cortex-M
- Dari ELF ke chip

Di Dalam Chip: CPU, Bus, dan Periferal

- ISA: kontrak antara compiler dan silikon
- Pipeline: instruksi tidak dikerjakan satu per satu
- Peta memori: semua hal punya alamat
- Dari instruksi STR ke pin yang menyala
- Interupsi dan DMA: hardware yang bergerak sendiri

Memory Alignment Secara Tuntas

Definisi dan intuisi
Mengapa hardware peduli: empat lapisan alasan
Peta perilaku per arsitektur
Bagaimana compiler menjaga alignment: padding struct
packed: mematikan padding, dan harganya
Cara yang benar membaca data tak-aligned
Menaikkan alignment: kapan dan mengapa
DMA, cache, dan alignment: jebakan terbesar di Cortex-M7
Daftar periksa alignment untuk firmware
Ketika Compiler dan Hardware Salah Paham
volatile: "jangan berasumsi tentang memori ini"
Barrier: dua jenis pagar
Strict aliasing: compiler percaya pada tipe
Endianness: urutan byte di dalam word
Bit-field: nyaman tetapi tidak portabel
Studi Kasus RFID: Dari Udara ke Server
Tiga frekuensi, tiga dunia
Sisi tag: chip tanpa CPU, tanpa compiler
Sisi pembaca: lapisan di dalam lapisan
Mengikuti satu EPC dari udara ke server
Data & Angka Kunci
Perbandingan arsitektur dan alignment
Aturan alignment ABI 32-bit (ringkas)
Angka-angka RFID
Perdebatan & Perspektif
Optimisasi berbasis undefined behavior: fitur atau jebakan?
Apakah volatile sudah cukup?
Bit-field dan struct packed untuk register dan protokol
C, C++, atau Rust untuk firmware?
Assembly tulisan tangan
Kondisi Terkini & Prospek
Kronologi
Glosarium
Kesimpulan
Bacaan Lanjutan

Pendahuluan

Setiap kali sebuah pembaca RFID di gudang "berbunyi" karena menangkap tag di sebuah kardus, ada rantai panjang yang bekerja tanpa terlihat. Seseorang menulis baris kode seperti `if (tag_ditemukan) kirim_ke_server(epc);`. Baris itu kemudian diterjemahkan, dioptimalkan, dipecah menjadi puluhan instruksi mesin, disusun di alamat-alamat tertentu di memori flash, lalu dieksekusi oleh jutaan transistor di sebuah mikrokontroler. Mikrokontroler itu berbicara dengan chip radio lewat kabel SPI, chip radio memancarkan gelombang 900 MHz, dan sebuah chip seukuran butiran pasir di dalam label tag menyerap energi gelombang itu, menghidupkan dirinya sendiri, lalu menjawab dengan "memantulkan" sinyal.

Di setiap sambungan rantai itu ada **kontrak**: aturan tentang bagaimana satu lapisan boleh memperlakukan lapisan di bawahnya. Bahasa C berjanji pada programmer tentang arti sebuah program. Compiler berjanji menghasilkan instruksi yang "berperilaku seolah-olah" sama. Arsitektur prosesor berjanji tentang apa yang terjadi ketika instruksi `STR` menulis ke alamat `0x40013804`. Dan chip periferal berjanji bahwa menulis ke alamat itu akan mengirimkan satu byte ke kabel serial. Sebagian besar bug paling sulit di sistem tertanam (*embedded*) lahir di **celah antar-kontrak** itu: compiler "membuang" pembacaan register karena merasa tidak perlu, prosesor menolak membaca angka 32-bit dari alamat ganjil, atau cache menyimpan data lama yang sebenarnya sudah ditimpa oleh DMA.

Dokumen ini mencoba menjawab pertanyaan-pertanyaan berikut:

1. **Apa yang sebenarnya dilakukan compiler**, tahap demi tahap, dari teks sumber sampai file biner yang di-*flash* ke chip?
2. **Bagaimana sebuah instruksi menjadi aksi fisik**: sinyal di bus, register periferal, dan perubahan tegangan di pin?
3. **Mengapa memory alignment penting**, dari mana aturan itu berasal (bus, cache, ABI), apa bedanya di x86, ARM Cortex-M, dan RISC-V, dan bagaimana menghindari jebakannya?
4. **Di titik mana compiler dan hardware bisa "salah paham"**, misalnya soal `volatile`, *memory barrier*, *strict aliasing*, *endianness*, *bit-field*, cache, dan DMA?
5. **Bagaimana semua itu terlihat di dunia RFID**, baik di sisi tag (chip pasif tanpa baterai) maupun di sisi pembaca (MCU + chip radio + protokol jaringan)?

RUANG LINGKUP DAN CARA MEMBACA

Topik ini sangat luas, jadi dokumen ini memilih satu "jalur utama": **bahasa C** (dengan sedikit C++ dan Rust), **compiler GCC/Clang-LLVM**, dan **mikrokontroler ARM Cortex-M**, karena kombinasi inilah yang paling umum dipakai di perangkat RFID, IoT, dan elektronik industri. x86 (PC/server) dan RISC-V dibahas sebagai pembanding. Detail fisika semikonduktor (cara transistor dibuat) dan desain compiler tingkat lanjut (teori parsing, *polyhedral optimization*) sengaja tidak dibahas mendalam. Contoh kode assembly bersifat ilustratif: hasil compiler nyata bisa sedikit berbeda tergantung versi dan opsi. Untuk melihatnya sendiri, coba situs *Compiler Explorer* (godbolt.org).

Pembaca yang ingin intinya saja cukup membaca **Gambaran Besar**, bab **Memory Alignment Secara Tuntas**, dan **Kesimpulan**. Pembaca dari dunia RFID bisa langsung melompat ke **Studi Kasus RFID** lalu kembali ke bab-bab sebelumnya saat menemukan istilah yang belum jelas. Semua istilah teknis dijelaskan di **Glosarium**.

Gambaran Besar

Tujuh lapisan dari ide ke elektron

Cara paling mudah memahami hubungan compiler dan hardware adalah membayangkan sebuah tangga. Di puncak ada gagasan manusia; di dasar ada elektron yang bergerak di dalam silikon. Setiap anak tangga menerjemahkan bentuk di atasnya menjadi bentuk yang lebih dekat ke mesin.

Lapisan	Bentuk "program" di lapisan ini	Siapa yang mengerjakan	Contoh
1. Bahasa sumber	Teks: fungsi, variabel, tipe	Programmer	`uint32_t epc_crc = hitung_crc(buf, 12);`
2. Representasi antara (IR)	Instruksi abstrak, register tak terbatas	*Front-end* dan *optimizer* compiler	`%3 = call i32 @hitung_crc(ptr %buf, i32 12)`
3. Assembly / kode mesin	Instruksi spesifik arsitektur	*Back-end* compiler, assembler	`movs r1, #12` lalu `bl hitung_crc`
4. File objek dan biner	Byte-byte di *section* `.text`, `.data`, `.bss`	Assembler, linker	`firmware.elf`, `firmware.bin`
5. Memori chip	Byte di flash dan SRAM pada alamat tertentu	*Programmer/debugger*, kode *startup*	Vektor reset di `0x0800_0004`
6. Mikroarsitektur	*Fetch*, *decode*, *execute*, transaksi bus	Inti CPU (misal Cortex-M4)	Pipeline 3 tahap, bus AHB
7. Periferal dan fisik	Register periferal, sinyal pin, gelombang RF	Blok SPI, UART, chip radio, antena	Clock SPI 8 MHz ke chip pembaca RFID

Yang membuat tangga ini menarik adalah bahwa **setiap lapisan hanya melihat kontrak lapisan di bawahnya, bukan isinya.** Programmer C tidak tahu (dan secara resmi tidak perlu tahu) apakah variabelnya disimpan di register `r4` atau di stack. Compiler tidak tahu bahwa alamat `0x4001_3804` adalah register data UART: baginya itu hanya alamat memori. CPU tidak tahu bahwa byte yang ia tulis akan menjadi sinyal radio. Abstraksi inilah yang membuat perangkat lunak modern bisa dibangun, tetapi juga sumber kebingungan ketika abstraksinya "bocor".

Tiga ide yang menjelaskan hampir semuanya

Pertama: compiler bekerja untuk "mesin abstrak", bukan untuk chip Anda. Standar bahasa C mendefinisikan arti program terhadap sebuah mesin imajiner. Compiler boleh mengubah apa pun (menghapus variabel, menyusun ulang perintah, menggabungkan pembacaan memori) selama perilaku yang *bisa diamati* menurut standar tetap sama. Masalahnya, menurut standar, membaca alamat memori biasa bukan "perilaku yang bisa diamati". Padahal di mikrokontroler, membaca register status UART bisa menghapus flag interupsi. Karena itulah ada kata kunci `volatile`, jembatan pertama antara dunia compiler dan dunia hardware.

Kedua: hardware punya "selera" tentang alamat. Memori fisik tidak dibaca per byte, tetapi per *word* (misalnya 4 byte) melalui bus, dan per *cache line* (32 atau 64 byte) melalui cache. Data yang "duduk rapi" di batas alamat kelipatan ukurannya (*aligned*) bisa diambil dalam satu kali jalan. Data yang menyeberangi batas itu harus diambil dua kali, atau, di prosesor sederhana seperti Cortex-M0, langsung memicu *HardFault*. Compiler mengetahui selera ini lewat dokumen **ABI** (*Application Binary Interface*), lalu menambahkan *padding* (byte kosong) di dalam struct supaya setiap field selalu aligned.

Ketiga: periferal adalah memori yang "hidup". Di arsitektur ARM Cortex-M, register-register periferal (GPIO, SPI, UART, timer) dipetakan ke ruang alamat yang sama dengan RAM. Menulis ke alamat tertentu sama dengan menekan tombol di periferal. Konsep ini disebut **memory-mapped I/O (MMIO)**. Karena dari sudut pandang instruksi tidak ada bedanya antara RAM dan register periferal, seluruh tanggung jawab untuk memperlakukannya dengan benar ada pada programmer dan compiler.

Fakta kunci

Hal	Fakta
Compiler pertama yang beroperasi	Diperdebatkan: A-0 Grace Hopper (UNIVAC I) dan Autocode Alick Glennie, keduanya 1952[1]
Compiler optimisasi pertama yang sukses	FORTRAN I untuk IBM 704, dirilis ke pelanggan April 1957[34]
Bahasa C	Lahir di Bell Labs 1969–1973, periode paling kreatif tahun 1972[5]
LLVM	Dimulai tahun 2000 di University of Illinois; rilis 1.0 tahun 2003[6]
Aturan alignment ABI Arm	Tipe primitif berukuran 1, 2, 4, 8 byte harus aligned sesuai ukurannya; stack 8-byte aligned di batas fungsi publik[17]
Cortex-M0/M0+	Tidak mendukung akses tak-aligned; mencobanya memicu HardFault[1920]
Cache line Cortex-M7	32 byte; buffer DMA idealnya aligned dan berukuran kelipatan 32[34]
Cache line x86 modern	64 byte; akses tak-aligned di dalam satu line praktis gratis, yang menyeberang line lebih mahal[2223]
Memori tag UHF Gen2	4 bank (Reserved, EPC, TID, User); dialamatkan per *word* 16-bit[4244]
Chip pembaca UHF Impinj E710	Antarmuka host SPI; di dalamnya ada MCU dengan MAC sendiri[49]

Sejarah Singkat: Dari Kabel ke Compiler

Ketika "memprogram" berarti memasang kabel

Komputer elektronik generasi pertama di tahun 1940-an tidak punya bahasa pemrograman. Program dimasukkan dengan memasang kabel, memutar sakelar, atau menulis kode mesin dalam bentuk angka. Setiap instruksi harus ditulis sebagai pola bit yang dipahami langsung oleh mesin, dan alamat memori dihitung dengan tangan. Kalau satu instruksi disisipkan di tengah program, semua alamat sesudahnya bergeser dan harus dihitung ulang. Masalah inilah yang kelak diselesaikan oleh *assembler* dan *linker*.

Langkah pertama keluar dari kesulitan itu adalah **pustaka subrutin**: potongan program yang sering dipakai (misalnya menghitung sinus) disimpan dan disalin ke program baru. Grace Hopper, yang bekerja dengan komputer UNIVAC I, mengumpulkan subrutin-subrutin itu di pita magnetik dan membuat sistem bernama **A-0** (1951–1952). Sistem ini mengambil subrutin yang diperlukan dan merangkainya menjadi program utuh[2]. Dalam istilah modern, A-0 lebih mirip *linker/loader* daripada compiler penuh, dan karena itu gelar "compiler pertama" masih diperdebatkan. Sebuah tulisan di blog *Communications of the ACM* mencatat bahwa Alick Glennie di Inggris juga mengklaim compiler pertama yang benar-benar berjalan di komputer, juga pada tahun 1952[1]. Yang tidak diperdebatkan adalah perlawanan yang dihadapi Hopper: banyak orang saat itu yakin komputer tidak bisa "menulis programnya sendiri".

FORTRAN: bukti bahwa mesin bisa menulis kode yang cepat

Keraguan terbesar terhadap bahasa tingkat tinggi bukanlah "apakah bisa", melainkan "apakah hasilnya cukup cepat". Waktu komputer sangat mahal, dan programmer assembly yang berpengalaman bisa memeras setiap siklus. Tim IBM yang dipimpin **John Backus** menjawab keraguan itu dengan **FORTRAN** (*FORmula TRANslation*). Proyek dimulai sekitar 1954, manual pertama terbit Oktober 1956, dan compiler-nya dikirim ke pelanggan IBM 704 pada April 1957, jauh melewati target awal enam bulan[4]. IBM sendiri menganggap 1957 sebagai "tanggal lahir" FORTRAN[3].

Pencapaian terpenting FORTRAN bukan sintaksnya, tetapi **optimizer-nya**. Compiler ini menganalisis program dan menghasilkan kode yang hampir secepat buatan tangan[4]. Inilah pertama kalinya terbukti bahwa sebuah program bisa memahami program lain cukup dalam untuk memperbaikinya. Gagasan-gagasan dari compiler FORTRAN, seperti analisis alur kontrol dan alokasi register, masih menjadi inti setiap compiler sampai hari ini.

C: bahasa yang "dekat dengan logam"

Bahasa C lahir di Bell Labs antara 1969 dan 1973, bersamaan dengan pengembangan sistem operasi Unix. Dennis Ritchie menulis bahwa periode paling kreatifnya terjadi pada 1972[5]. C adalah keturunan bahasa B, yang oleh Ritchie digambarkan sebagai BCPL yang "diperas" agar muat di memori 8 KB[5]. Tujuan C sangat praktis: menulis sistem operasi yang bisa dipindahkan ke mesin lain tanpa ditulis ulang dalam assembly.

Desain C mencerminkan hardware zamannya. Pointer adalah alamat memori. Array adalah deretan byte yang berdekatan. `struct` adalah cara memberi nama pada potongan-potongan memori. Tidak ada pengecekan batas array dan tidak ada *garbage collector*. Kedekatan inilah yang membuat C, lima puluh tahun kemudian, masih menjadi bahasa utama untuk firmware mikrokontroler, termasuk hampir semua firmware pembaca RFID. Namun kedekatan itu juga ilusi yang berbahaya: seperti yang akan kita lihat, compiler C modern **tidak** menerjemahkan kode baris per baris secara harfiah. C distandardisasi oleh komite ANSI X3J11 pada pertengahan 1980-an[5], lalu oleh ISO (C90, C99, C11, C17, dan C23).

GCC dan LLVM: dua raksasa compiler terbuka

Selama tahun 1980–1990-an, compiler untuk mikroprosesor umumnya produk komersial dari vendor chip atau perusahaan seperti Keil dan IAR. Proyek GNU melahirkan **GCC** (GNU Compiler Collection) pada akhir 1980-an. GCC kemudian menjadi compiler standar dunia Linux dan, lewat varian `arm-none-eabi-gcc`, compiler paling umum untuk ARM Cortex-M.

Tahun 2000, Chris Lattner dan Vikram Adve memulai proyek **LLVM** di University of Illinois at Urbana-Champaign[6]. Ide intinya adalah membuat compiler yang modular: sebuah *representasi antara* (IR) yang terdefinisi rapi, sehingga *front-end* untuk berbagai bahasa dan *back-end* untuk berbagai chip bisa dicampur dan dicocokkan. Makalah mereka di GCC Summit 2003 menyebutkan rencana rilis publik pertama dengan lisensi longgar pada musim panas 2003[7]; LLVM 1.0 diumumkan pada akhir Oktober 2003[6]. Hari ini LLVM menjadi fondasi Clang (C/C++), Rust, Swift, dan banyak compiler lain, termasuk toolchain resmi Arm.

> **MENGAPA SEJARAH INI RELEVAN UNTUK INSINYUR RFID**
>
> Firmware pembaca RFID yang Anda tulis hari ini kemungkinan besar dikompilasi oleh GCC (lewat STM32CubeIDE, ESP-IDF, atau Zephyr) atau oleh compiler berbasis LLVM (Arm Compiler 6, Rust). Kedua compiler ini mewarisi filosofi FORTRAN: **compiler adalah optimizer yang agresif.** Ia tidak bertugas menjalankan perintah Anda kata demi kata, melainkan menghasilkan program tercepat yang perilakunya "sama" menurut standar bahasa. Memahami kata "sama" di sini adalah kunci seluruh dokumen.

Bahasa Pemrograman sebagai Kontrak

Mesin abstrak dan aturan "seolah-olah"

Standar C tidak mendefinisikan arti program dalam bentuk instruksi ARM atau x86. Ia mendefinisikan sebuah **mesin abstrak**, yaitu komputer imajiner tempat setiap ekspresi dievaluasi persis sesuai teks. Lalu standar memberi compiler kebebasan besar lewat aturan yang biasa disebut ***as-if rule***: implementasi boleh melakukan apa saja asalkan **perilaku yang bisa diamati** dari program sama dengan perilaku mesin abstrak.

Apa saja yang termasuk "bisa diamati"? Secara garis besar hanya tiga: akses ke objek `volatile`, data yang ditulis ke file atau keluar program, dan interaksi dengan perangkat interaktif. Selebihnya, yaitu nilai variabel biasa, urutan perhitungan, berapa kali memori dibaca, adalah urusan dalam compiler. Contoh sederhana:

```
int jumlah_tag(void) {
    int total = 0;
    for (int i = 0; i < 100; i++)
        total += 2;
    return total;
}
```

Compiler dengan optimisasi tidak akan membuat loop sama sekali. Ia langsung menghasilkan `return 200;`, karena hasil akhirnya sama. Ini contoh yang tidak berbahaya. Contoh yang berbahaya muncul ketika "memori biasa" ternyata adalah register hardware:

```
#define UART_SR  (*(uint32_t *)0x40013800u)   /* register status, TANPA volatile */
#define RXNE     (1u << 5)

void tunggu_byte(void) {
    while ((UART_SR & RXNE) == 0) { }   /* tunggu sampai ada data masuk */
}
```

Menurut mesin abstrak, tidak ada yang mengubah nilai di alamat `0x40013800` di dalam loop. Jadi compiler boleh membaca alamat itu **sekali**, lalu, kalau bitnya 0, berputar selamanya tanpa membaca lagi. Di dunia nyata, hardware UART mengubah bit itu ketika byte datang, tetapi CPU tidak pernah melihatnya. Program "macet" hanya ketika dikompilasi dengan `-O2`, dan berjalan normal dengan `-O0`. Pola bug "jalan di *debug*, macet di *release*" ini sangat khas di firmware.

Tiga jenis "ketidakpastian" yang diizinkan standar

Standar C sengaja tidak mendefinisikan semua hal, supaya bahasa bisa berjalan efisien di hardware yang sangat beragam. Ada tiga kategori yang harus dibedakan:

Kategori	Arti	Contoh yang relevan dengan hardware
Implementation-defined	Compiler bebas memilih, tetapi **wajib mendokumentasikan** pilihannya	Ukuran `int`, apakah `char` bertanda, urutan bit dalam *bit-field*
Unspecified	Compiler bebas memilih di antara beberapa kemungkinan, tanpa wajib dokumentasi	Urutan evaluasi argumen fungsi `f(a(), b())`
***Undefined behavior* (UB)**	Standar tidak memberi jaminan apa pun; compiler boleh menganggap ini **tidak pernah terjadi**	Membaca `uint32_t` lewat pointer tak-aligned, *signed overflow*, melanggar *strict aliasing*, mengakses di luar batas array

Kategori ketiga adalah yang paling sering disalahpahami. *Undefined behavior* bukan berarti "hasilnya acak". Yang lebih berbahaya, compiler **memakai asumsi bahwa UB tidak pernah terjadi** untuk mengoptimalkan kode. Kalau programmer menulis `*(uint32_t *)(buf + 3)`, compiler berhak menganggap alamat itu pasti kelipatan 4, lalu memilih instruksi yang hanya benar untuk alamat aligned. Di Cortex-M4 mungkin tetap jalan; di Cortex-M0 langsung *HardFault*; dan di kasus lain bisa muncul nilai yang salah tanpa tanda apa pun. Kita akan kembali ke contoh ini di bab alignment dan studi kasus RFID.

Tipe data: bahasa yang berbicara tentang ukuran dan alamat

Setiap tipe di C membawa tiga informasi yang langsung berhubungan dengan hardware:

1. **Ukuran** (`sizeof`): berapa byte yang ditempati.
2. **Alignment** (`alignof`, sejak C11): alamat objek tipe itu harus kelipatan berapa.
3. **Representasi**: bagaimana bit-bitnya diartikan (bilangan bulat komplemen dua, IEEE-754, pointer).

C11 menambahkan operator `_Alignof` dan penentu `_Alignas`, yang biasa dipakai lewat makro `alignof` dan `alignas` dari `<stdalign.h>`[14]; di C23 keduanya menjadi kata kunci biasa. Usulan resmi WG14 yang melatarbelakanginya (dokumen N1507) bertujuan menyamakan cara C dan C++ berbicara tentang alignment[15]. Sebelum C11, programmer harus memakai ekstensi compiler seperti `__attribute__((aligned(8)))` di GCC atau `__align` di compiler Arm lama.

Tipe berukuran tetap seperti `uint8_t`, `uint16_t`, dan `uint32_t` (dari `<stdint.h>`, sejak C99) sangat penting di firmware. `int` di mikrokontroler 8-bit AVR berukuran 16 bit, sedangkan di

Cortex-M 32 bit. Kalau Anda mendefinisikan format paket protokol RFID dengan `int`, ukurannya bisa berubah ketika kode dipindahkan ke chip lain.

Di Dalam Compiler

Peta besar: front-end, middle-end, back-end

Compiler modern hampir selalu dibagi tiga[8]:

- **Front-end** memahami bahasa sumber: membaca teks, memeriksa tata bahasa dan tipe, lalu menurunkannya ke IR.
- **Middle-end (optimizer)** mengubah IR menjadi IR lain yang lebih efisien. Karena bekerja di IR, optimizer ini bisa dipakai bersama oleh banyak bahasa.
- **Back-end** menerjemahkan IR ke instruksi chip tertentu: memilih instruksi, membagi register, menyusun ulang urutan, dan menghasilkan assembly atau kode objek.

Pembagian ini adalah alasan mengapa satu compiler bisa melayani puluhan arsitektur. Di LLVM, kode khusus target berada di direktori `llvm/lib/Target`, misalnya `ARM`, `RISCV`, dan `X86`, sementara optimizer-nya sama untuk semuanya. Akibat praktisnya: compiler mengetahui hal-hal spesifik chip (berapa register, alignment apa yang diizinkan, instruksi apa yang tersedia) **hanya** lewat deskripsi target dan opsi seperti `-mcpu=cortex-m0plus`. Kalau opsi ini salah, compiler akan dengan percaya diri menghasilkan instruksi yang tidak dimiliki chip Anda.

Tahap 1: dari teks ke pohon (lexing, parsing, analisis semantik)

Ambil fungsi kecil dari firmware pembaca RFID ini:

```
uint16_t ambil_pc(const uint8_t *resp) {
    return (uint16_t)((resp[0] << 8) | resp[1]);   /* PC word, big-endian */
}
```

Preprocessor berjalan lebih dulu: ia mengganti `#include` dengan isi file header, mengembangkan makro, dan memproses `#if`. Hasilnya masih teks.

Lexer memotong teks menjadi *token*: `uint16_t`, `ambil_pc`, `(`, `const`, `uint8_t`, `*`, `resp`, dan seterusnya. Spasi dan komentar dibuang.

Parser menyusun token menjadi **pohon sintaks abstrak** (AST). Ekspresi `(resp[0] << 8) | resp[1]` menjadi sebuah simpul `|` dengan anak kiri `<<` (yang punya anak `resp[0]` dan `8`) dan anak kanan `resp[1]`.

Analisis semantik memeriksa makna: apakah `resp` dideklarasikan, apa tipenya, dan konversi apa yang terjadi. Di sinilah aturan C yang sering mengejutkan diterapkan. `resp[0]` bertipe `uint8_t`, tetapi sebelum digeser ia **dipromosikan** menjadi `int` (*integer promotion*). Karena itu `resp[0] <<`

`8` aman. Seandainya kita menulis `resp[0] << 24` dan `resp[0]` bernilai 0x80 atau lebih, hasilnya melampaui batas `int` bertanda, yang merupakan UB. Inilah sebabnya kode parsing protokol yang benar menulis `(uint32_t)resp[0] << 24`.

Tahap 2: representasi antara dan bentuk SSA

Setelah lolos front-end, program diturunkan ke **IR**. LLVM IR, misalnya, ada dalam tiga bentuk yang setara: teks `.ll` yang bisa dibaca manusia, *bitcode* biner `.bc`, dan struktur data di memori[8]. Fungsi di atas kira-kira menjadi:

```
define i16 @ambil_pc(ptr %resp) {
  %b0  = load i8, ptr %resp, align 1
  %p1  = getelementptr i8, ptr %resp, i32 1
  %b1  = load i8, ptr %p1, align 1
  %z0  = zext i8 %b0 to i32
  %sh  = shl i32 %z0, 8
  %z1  = zext i8 %b1 to i32
  %or  = or i32 %sh, %z1
  %res = trunc i32 %or to i16
  ret i16 %res
}
```

Perhatikan tiga hal. **Pertama**, setiap nilai hanya ditetapkan sekali (`%b0`, `%sh`, ...). Bentuk ini disebut **SSA** (*Static Single Assignment*) dan membuat analisis aliran data jauh lebih mudah. **Kedua**, IR menganggap register tidak terbatas; pembagian ke register fisik baru dilakukan nanti. **Ketiga**, dan ini penting untuk topik kita, **setiap `load` membawa informasi alignment** (`align 1`). Compiler mencatat apa yang ia ketahui tentang alamat. Seandainya kodenya `*(uint32_t *)resp`, IR-nya akan berbunyi `load i32, ptr %resp, align 4`: compiler **menjanjikan** kepada back-end bahwa alamatnya kelipatan 4, karena itulah alignment `uint32_t` menurut ABI. Janji itulah yang dilanggar ketika programmer melakukan *cast* sembarangan.

Tahap 3: optimisasi

Optimizer LLVM dan GCC terdiri dari ratusan *pass* kecil, masing-masing melakukan satu transformasi, dan disusun berbeda untuk `-O0`, `-O1`, `-O2`, `-O3`, `-Os`, dan `-Oz`. Beberapa yang paling penting untuk dipahami insinyur embedded:

Optimisasi	Apa yang dilakukan	Dampak ke hardware
Constant folding & propagation	Menghitung ekspresi konstan saat kompilasi	`0x40000000 + 0x3800` menjadi satu konstanta alamat
Dead code elimination	Membuang perhitungan yang hasilnya tidak dipakai	Pembacaan register non-`volatile` yang hasilnya tak dipakai **hilang**
Common subexpression elimination	Menghitung ekspresi yang sama sekali saja	Dua pembacaan register yang sama bisa digabung jadi satu
Loop-invariant code motion	Mengangkat perhitungan yang tetap ke luar loop	Pembacaan flag status di dalam loop bisa diangkat keluar (bug polling)
Inlining	Menyalin isi fungsi kecil ke pemanggil	Kode lebih cepat tapi lebih besar; alur kode di debugger berubah
Store merging / load widening	Menggabungkan beberapa akses kecil menjadi satu akses lebar	Empat tulisan byte bisa jadi satu tulisan 32-bit, **hanya jika** alignment diketahui aman
Vectorization	Memproses beberapa elemen sekaligus dengan instruksi SIMD	Di Cortex-M4/M7 (DSP) dan x86 (SSE/AVX); sangat sensitif alignment
Alias analysis	Menentukan apakah dua pointer bisa menunjuk ke memori yang sama	Dasar dari aturan *strict aliasing*

Sebagai contoh, fungsi `ambil_pc` di atas, dengan `-O2` untuk Cortex-M4, bisa dikompilasi menjadi beberapa instruksi saja: dua pembacaan byte, satu penggabungan, dan kembali. Di x86-64, Clang bahkan bisa mengenali pola "baca 16 bit lalu tukar urutan byte", lalu memakai satu pembacaan 16-bit diikuti instruksi `rol` / `bswap`, karena di x86 pembacaan tak-aligned diizinkan. Untuk Cortex-M0, compiler **tidak** boleh melakukan pelebaran itu, karena alamat `resp` hanya dijamin aligned 1 byte dan M0 tidak bisa membaca halfword dari alamat ganjil. Jadi kode C yang sama menghasilkan strategi berbeda, dan keduanya benar, karena compiler tahu kemampuan masing-masing target.

Tahap 4: back-end, dari IR ke instruksi nyata

Back-end bekerja dalam beberapa langkah besar:

Pemilihan instruksi (*instruction selection*). Pola-pola IR dicocokkan dengan instruksi yang tersedia. `shl` lalu `or` di ARM bisa menjadi satu instruksi `orr r0, r1, r0, lsl #8`, karena ARM bisa menggeser operan kedua tanpa biaya tambahan. Di sinilah compiler memakai pengetahuan tentang alignment: `load i32 ... align 4` boleh menjadi `LDR`, sedangkan `load i32 ... align 1` di Cortex-M0 harus dipecah menjadi empat `LDRB` plus penggeseran dan penggabungan.

Alokasi register. Cortex-M punya 13 register serbaguna (`r0` – `r12`) ditambah `sp` , `lr` , dan `pc` . Ribuan nilai SSA harus dipetakan ke register yang terbatas itu. Nilai yang tidak kebagian register di-*spill* ke stack. Konvensi pemanggilan **AAPCS** (*Procedure Call Standard for the Arm Architecture*) menentukan bahwa argumen pertama sampai keempat masuk di `r0` – `r3` , nilai kembali di `r0` , dan register `r4` – `r11` harus dipulihkan oleh fungsi yang memakainya. Aturan seperti ini memungkinkan kode yang dikompilasi oleh compiler berbeda (misalnya pustaka vendor buatan IAR dan aplikasi Anda buatan GCC) saling memanggil.

Penjadwalan instruksi (*scheduling*). Instruksi diurutkan ulang supaya pipeline prosesor tidak menganggur, misalnya memisahkan instruksi `LDR` dari instruksi yang memakai hasilnya.

Emisi kode. Hasil akhirnya berupa file assembly (`.s`) atau langsung file objek (`.o`). Hasil akhir untuk `ambil_pc` di Cortex-M4 dengan `-O2` kira-kira:

```
ambil_pc:
    ldrb    r1, [r0]          @ r1 = resp[0]
    ldrb    r0, [r0, #1]      @ r0 = resp[1]
    orr     r0, r0, r1, lsl #8
    bx      lr                @ kembali; hasil di r0
```

Empat instruksi, sebagian besar 16-bit. Dari sebuah ekspresi C dengan promosi tipe, *cast*, dan dua akses array, compiler menyisakan pekerjaan yang benar-benar dibutuhkan hardware.

MELIHAT SENDIRI APA YANG DILAKUKAN COMPILER

- `arm-none-eabi-gcc -O2 -mcpu=cortex-m4 -mthumb -S file.c` menghasilkan `file.s` berisi assembly.
- `clang --target=thumbv6m-none-eabi -O2 -S -emit-llvm file.c` menghasilkan LLVM IR.
- `arm-none-eabi-objdump -d firmware.elf` membongkar biner yang sudah jadi.
- `-fdump-tree-optimized` (GCC) memperlihatkan representasi antara GCC setelah optimisasi.
- Situs Compiler Explorer (godbolt.org) menampilkan kode C dan assembly berdampingan untuk puluhan compiler dan target, sangat berguna untuk membandingkan `cortex-m0plus` dan `cortex-m4` .

Mengapa level optimisasi mengubah perilaku program

Programmer firmware sering curiga ketika bug muncul hanya di `-O2` : "compiler-nya bug." Sangat jarang begitu. Yang lebih sering terjadi, program memang mengandung UB atau lupa `volatile` , dan `-O0` kebetulan menyembunyikannya karena di level itu compiler menyimpan setiap variabel di memori dan membacanya ulang setiap kali dipakai. Dengan kata lain, `-O0` menghasilkan kode yang

mendekati terjemahan "harfiah", sedangkan `-O2` memanfaatkan setiap kebebasan yang diberikan standar. Prinsip praktisnya: **uji firmware dengan level optimisasi yang sama dengan rilis**, dan perlakukan "jalan di `-O0`, gagal di `-O2`" sebagai tanda ada kontrak yang dilanggar, bukan alasan untuk menurunkan optimisasi.

Assembler, Linker, dan Kode Startup

File objek: kode yang belum tahu alamatnya

Assembler mengubah teks assembly menjadi **file objek** (`.o`), biasanya berformat **ELF** di dunia GCC/LLVM. File objek berisi beberapa *section*[3940]:

Section	Isi	Di mana akhirnya disimpan di mikrokontroler
`.text`	Kode mesin	Flash
`.rodata`	Konstanta (`const`), string, tabel CRC	Flash
`.data`	Variabel global/statis yang punya nilai awal bukan nol	Disimpan di flash, **disalin** ke RAM saat startup
`.bss`	Variabel global/statis bernilai awal nol	Hanya di RAM; **diisi nol** saat startup; tidak memakan tempat di file
`.stack`, `.heap`	Ruang stack dan heap (ditentukan linker script)	RAM

Yang menarik, setiap section di file objek dimulai dari alamat nol[40]. Ketika fungsi `baca_tag()` di `rfid.o` memanggil `spi_transfer()` yang ada di `spi.o`, assembler belum tahu alamat `spi_transfer`. Ia hanya meninggalkan "lubang" di instruksi `BL` dan mencatat **entri relokasi**: "di offset sekian dalam `.text`, isi dengan alamat simbol `spi_transfer`, memakai rumus jenis `R_ARM_THM_CALL`." Tabel simbol (`.symtab`) mencatat simbol apa yang didefinisikan file ini dan simbol apa yang dibutuhkannya dari luar.

Linker: menyusun peta memori

Linker (`ld`, `lld`, atau `gold`) punya dua tugas utama[39]:

1. **Resolusi simbol**: setiap referensi ke `spi_transfer` dihubungkan ke tepat satu definisi. Kalau tidak ada definisi, muncul galat *undefined reference*; kalau ada dua, muncul *multiple definition*.
2. **Relokasi**: menentukan alamat akhir setiap section, lalu menambal semua "lubang" yang dicatat assembler.

Di PC, linker menghasilkan program yang nanti dimuat oleh sistem operasi ke alamat virtual. Di mikrokontroler tidak ada sistem operasi yang memuat program; **alamat fisik harus ditentukan saat link**. Untuk itu ada **linker script** (`.ld`), file yang menjelaskan peta memori chip:

```
MEMORY
{
  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 256K
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 64K
}

SECTIONS
{
  .isr_vector : { KEEP(*(.isr_vector)) } > FLASH   /* tabel vektor di awal flash */
  .text       : { *(.text*) *(.rodata*) } > FLASH
  .data       : { _sdata = .; *(.data*); _edata = .; } > RAM AT > FLASH
  _sidata = LOADADDR(.data);                       /* lokasi salinan .data di flash */
  .bss  (NOLOAD) : { _sbss = .; *(.bss*) *(COMMON); _ebss = .; } > RAM
  . = ALIGN(8);
  _estack = ORIGIN(RAM) + LENGTH(RAM);             /* stack tumbuh turun dari ujung RAM
*/
}
```

Baris `> RAM AT > FLASH` adalah inti cara kerja variabel global di mikrokontroler: alamat **eksekusi** (VMA) `.data` ada di RAM, tetapi **isinya** (LMA) disimpan di flash, karena RAM kehilangan isinya ketika listrik mati. Simbol `_sdata`, `_edata`, `_sidata`, `_sbss`, dan `_ebss` tidak berisi data; mereka hanya **label alamat** yang nanti dibaca oleh kode startup[38]. Linker script juga penuh perintah `ALIGN(...)`, karena tabel vektor, stack, dan buffer DMA punya syarat alignment sendiri. Linker script pustaka Rust `cortex-m-rt`, misalnya, memeriksa secara eksplisit bahwa tabel vektor diletakkan dengan alignment yang benar[37].

Saat listrik menyala: urutan reset Cortex-M

Apa yang terjadi di nanodetik pertama setelah tombol reset dilepas? Di Cortex-M, urutannya ditentukan oleh hardware, bukan oleh perangkat lunak[38]:

1. CPU membaca **word pertama** tabel vektor (misalnya di `0x0800_0000` pada STM32) dan memasukkannya ke register `SP` sebagai nilai awal stack.
2. CPU membaca **word kedua** (`0x0800_0004`), yaitu alamat `Reset_Handler`, lalu melompat ke sana.
3. `Reset_Handler` (kode startup, biasanya `startup_stm32xxx.s` atau ditulis dalam C) melakukan:
   - menyalin `.data` dari flash (`_sidata`) ke RAM (`_sdata` sampai `_edata`);
   - mengisi nol `.bss` (`_sbss` sampai `_ebss`);
   - (opsional) mengaktifkan FPU, mengatur clock sistem lewat `SystemInit()`;

- memanggil konstruktor C++ statis, lalu `main()`.

```
extern uint32_t _sidata, _sdata, _edata, _sbss, _ebss;

void Reset_Handler(void) {
    uint32_t *src = &_sidata, *dst = &_sdata;
    while (dst < &_edata) *dst++ = *src++;   /* salin .data */
    for (dst = &_sbss; dst < &_ebss; ) *dst++ = 0;  /* nolkan .bss */
    SystemInit();
    main();
    for (;;) { }
}
```

Perhatikan bahwa loop di atas menyalin **per word 32-bit**. Ini hanya benar kalau linker script menjamin `_sdata`, `_edata`, `_sbss`, dan `_ebss` aligned 4 byte. Kalau ada yang lupa `ALIGN(4)`, beberapa byte terakhir `.data` tidak tersalin atau byte di luar `.bss` ikut terhapus. Ini contoh nyata bagaimana alignment bukan hanya soal kecepatan, tetapi soal **kebenaran**.

> **KONSEKUENSI BAGI C: MENGAPA VARIABEL GLOBAL PUNYA NILAI AWAL**
>
> Standar C menjamin variabel statis tanpa inisialisasi bernilai nol. Di PC, jaminan ini dipenuhi oleh sistem operasi, yang memberikan halaman memori bersih. Di mikrokontroler, jaminan itu dipenuhi oleh **beberapa baris kode startup** di atas. Kalau kode startup dimodifikasi dan loop `.bss` terhapus, program C yang "benar" bisa berperilaku acak karena variabel global berisi sisa data dari sebelum reset.

Dari ELF ke chip

Hasil akhir linker adalah `firmware.elf`, yang masih membawa simbol dan informasi debug (berguna untuk GDB). Untuk ditulis ke flash, file itu sering diubah menjadi gambar biner mentah (`arm-none-eabi-objcopy -O binary`) atau format Intel HEX. *Programmer/debugger* (ST-LINK, J-Link, CMSIS-DAP) kemudian menulis byte-byte itu ke flash lewat antarmuka **SWD** atau **JTAG**. Di titik ini program kita sudah sepenuhnya berupa pola bit di sel-sel memori flash, siap dieksekusi.

Di Dalam Chip: CPU, Bus, dan Periferal

ISA: kontrak antara compiler dan silikon

Instruction Set Architecture (ISA) adalah kontrak paling penting dalam dokumen ini. Ia mendefinisikan instruksi apa yang ada, register apa yang terlihat, bagaimana alamat memori dihitung, dan apa yang terjadi pada akses tak-aligned. Compiler hanya perlu menaati ISA; ia tidak perlu tahu bagaimana ISA itu diwujudkan di silikon. Dua chip dengan ISA sama (misalnya Cortex-M4 buatan ST dan buatan NXP) bisa menjalankan biner yang sama untuk bagian CPU-nya, meskipun periferalnya berbeda.

Keluarga Cortex-M hanya mendukung instruksi **Thumb/Thumb-2**[31]. Thumb-2 adalah *encoding* campuran: sebagian besar instruksi berukuran 16 bit, dan sebagian lain 32 bit. Hasilnya adalah **kepadatan kode** yang tinggi, yang penting karena flash mikrokontroler mahal dan kecil. Harganya: banyak instruksi 16-bit hanya bisa memakai register `r0` – `r7`, dan eksekusi bersyarat memerlukan instruksi awalan `IT`[31]. Compiler mengetahui semua batasan ini dan memilih bentuk 16-bit setiap kali memungkinkan, terutama dengan `-Os`.

Inti	Arsitektur	Akses tak-aligned	Cache	Catatan
Cortex-M0 / M0+	ARMv6-M	**Tidak**; selalu *HardFault*[19][20]	Tidak	Paling kecil dan hemat daya; banyak di modul RFID murah
Cortex-M3	ARMv7-M	Ya untuk `LDR` / `STR` / `LDRH` / `STRH` biasa; tidak untuk `LDM` / `STM` / `LDRD`[60]	Tidak	Pipeline 3 tahap, bus Harvard[29]
Cortex-M4	ARMv7E-M	Sama seperti M3	Tidak (kadang ada *accelerator* flash dari vendor)	Instruksi DSP dan FPU opsional
Cortex-M7	ARMv7E-M	Sama seperti M3	**Ya**, cache data dengan line 32 byte[34]	Pipeline superskalar 6 tahap; perlu perhatian khusus untuk DMA
Cortex-M33 / M55 / M85	ARMv8-M	Sama seperti M3	M55/M85 ya	TrustZone; M55/M85 dengan vektor Helium

Pipeline: instruksi tidak dikerjakan satu per satu

Cortex-M3 memiliki pipeline tiga tahap: **fetch** (ambil instruksi dari memori), **decode** (pahami instruksinya), dan **execute** (kerjakan)[30]. Saat satu instruksi dieksekusi, instruksi berikutnya sedang di-*decode* dan instruksi sesudahnya sedang diambil. Arm menjelaskan bahwa Cortex-M3 bahkan mulai mengambil instruksi tujuan sebuah *branch* sudah di tahap *decode*, sebelum *branch*-nya benar-benar diselesaikan di tahap *execute*[29].

Yang lebih penting bagi pembahasan kita: Cortex-M3 memakai **arsitektur bus Harvard** dengan tiga antarmuka AHB-Lite terpisah, yaitu **I-Code** (ambil instruksi dari flash), **D-Code** (ambil data/konstanta dari flash), dan **System** (akses SRAM dan periferal)[30]. Karena jalurnya terpisah, CPU bisa mengambil instruksi berikutnya sambil membaca data, tanpa saling menunggu.

Peta memori: semua hal punya alamat

Arsitektur ARMv7-M membagi ruang alamat 32-bit (4 GB) menjadi wilayah-wilayah dengan fungsi tetap[27][28]:

Rentang alamat	Wilayah	Tipe memori	Isi tipikal
`0x0000_0000` – `0x1FFF_FFFF`	Code	Normal	Flash, tabel vektor
`0x2000_0000` – `0x3FFF_FFFF`	SRAM	Normal	Variabel, stack, heap, buffer DMA
`0x4000_0000` – `0x5FFF_FFFF`	Peripheral	**Device**, tidak boleh dieksekusi (XN)	GPIO, SPI, UART, timer, DMA
`0x6000_0000` – `0x9FFF_FFFF`	RAM eksternal	Normal	SDRAM, PSRAM
`0xA000_0000` – `0xDFFF_FFFF`	Perangkat eksternal	Device	Memori atau chip eksternal yang dipetakan
`0xE000_0000` – `0xE00F_FFFF`	Private Peripheral Bus	*Strongly-ordered*	NVIC, SysTick, System Control Block[28]

Tipe memori di kolom keempat bukan sekadar label. Ia mengubah aturan main CPU. Memori **Normal** boleh dibaca secara spekulatif, aksesnya boleh digabung atau disusun ulang, dan akses tak-aligned diizinkan (di M3 ke atas). Memori **Device** dan ***Strongly-ordered*** dianggap punya efek samping: setiap akses harus benar-benar terjadi, dalam urutan dan ukuran yang ditulis program. Tipe ini dirancang persis untuk register periferal. Beberapa inti juga menyediakan *bit-banding* (misalnya di Cortex-M3/M4): satu word di wilayah alias `0x4200_0000` memetakan satu bit di

wilayah periferal `0x4000_0000`[28], sehingga sebuah bit bisa diubah tanpa *read-modify-write*. Fitur ini opsional dan tidak ada di semua inti.

Dari instruksi `STR` ke pin yang menyala

Bagaimana sebuah tulisan C benar-benar menyalakan LED? Mari ikuti satu baris:

```
#define GPIOA_ODR (*(volatile uint32_t *)0x48000014u)

void led_nyala(void) {
    GPIOA_ODR |= (1u << 5);
}
```

Compiler menghasilkan kira-kira:

```
led_nyala:
    ldr     r2, =0x48000014   @ alamat register (diambil dari literal pool di flash)
    ldr     r3, [r2]          @ BACA nilai register sekarang
    orr     r3, r3, #32       @ set bit 5
    str     r3, [r2]          @ TULIS kembali
    bx      lr
```

Lalu di dalam chip:

1. **Fetch**: inti mengambil instruksi `str` dari flash lewat bus I-Code.
2. **Execute**: unit *load/store* mengirimkan transaksi **tulis 32-bit** ke bus System dengan alamat `0x4800_0014` dan data di `r3`.
3. **Matriks bus** (*bus matrix*) melihat bit-bit atas alamat dan meneruskan transaksi ke bus periferal yang tepat. Di banyak chip, periferal cepat ada di AHB dan periferal lambat di APB, dihubungkan lewat jembatan (*bridge*).
4. **Dekoder alamat** di blok GPIO memilih register `ODR` (offset `0x14`), lalu menyimpan nilai ke *flip-flop* register itu.
5. Keluaran flip-flop bit 5 terhubung ke *driver* pin. Tegangan pin PA5 naik ke 3,3 V dan LED menyala.

Inilah esensi MMIO: **dari sudut pandang CPU, menyalakan LED sama dengan menyimpan angka ke memori.** Seluruh "makna" ada di hardware di ujung bus.

TIGA JEBAKAN YANG TERSEMBUNYI DI BARIS SEDERHANA ITU

1. **Read-modify-write tidak atomik.** Jika sebuah interupsi mengubah bit lain di `GPIOA_ODR` di antara `ldr` dan `str`, perubahan itu tertimpa. Karena itu banyak chip menyediakan register khusus set/reset (misalnya `BSRR` di STM32) yang cukup ditulis tanpa dibaca.
2. **Clock periferal.** Di kebanyakan mikrokontroler, blok periferal mati sampai clock-nya diaktifkan (misalnya lewat register RCC). Tulisan ke register periferal yang clock-nya mati bisa diabaikan begitu saja. Compiler tidak tahu apa-apa soal ini.
3. **Lebar akses.** Sebagian register hanya menerima akses 32-bit. Kalau compiler (atau programmer) menulisnya dengan `STRB` 8-bit, hasilnya bisa diabaikan atau berbeda. `volatile uint32_t` memberi tahu compiler lebar aksesnya, tetapi sebuah laporan lama di milis GCC menunjukkan bahwa ini dulu pun pernah menjadi masalah[61].

Interupsi dan DMA: hardware yang bergerak sendiri

Dua mekanisme membuat hardware tidak sekadar "pasif menunggu perintah":

Interupsi. Ketika UART menerima byte atau chip RFID menarik pin IRQ, **NVIC** (*Nested Vectored Interrupt Controller*) menghentikan program utama, menyimpan beberapa register secara otomatis ke stack, lalu melompat ke alamat *handler* yang tercatat di tabel vektor. Fungsi interupsi di Cortex-M bisa ditulis sebagai fungsi C biasa, karena hardware menyimpan register sesuai konvensi AAPCS. Inilah contoh indah bagaimana hardware dirancang **untuk** compiler: arsitek Cortex-M sengaja membuat urutan masuk interupsi cocok dengan konvensi pemanggilan C. Satu syarat: stack harus aligned 8 byte di batas fungsi publik menurut AAPCS[17], dan hardware Cortex-M bisa menambahkan padding otomatis saat masuk interupsi untuk menjaga syarat itu.

DMA (*Direct Memory Access*). Pengendali DMA adalah "CPU kecil" yang hanya bisa memindahkan data, misalnya dari register data SPI ke buffer di SRAM, tanpa melibatkan CPU. Untuk pembaca RFID yang menerima ratusan respons tag per detik, DMA sangat berharga. Tetapi DMA juga membuat **dua pihak** mengubah memori yang sama tanpa saling tahu, dan compiler hanya tahu satu di antaranya. Ini dibahas lebih dalam di bab berikutnya.

Memory Alignment Secara Tuntas

Definisi dan intuisi

Sebuah alamat disebut **aligned ke N** jika alamat itu kelipatan N. Alamat `0x2000_0008` aligned ke 1, 2, 4, dan 8; alamat `0x2000_0006` aligned ke 2 tetapi tidak ke 4. Setiap tipe data punya **syarat alignment**: di kebanyakan ABI 32-bit, `uint16_t` harus aligned 2, `uint32_t` dan `float` harus aligned 4, dan `uint64_t` serta `double` aligned 8 (di ABI Arm) atau 4 (di beberapa ABI lain). ABI Arm merumuskannya dengan sederhana: tipe primitif berukuran 1, 2, 4, atau 8 byte harus aligned sesuai ukurannya sendiri[17]. Inilah yang disebut ***natural alignment***.

Bayangkan memori sebagai rak buku dengan sekat setiap 4 buku. Petugas perpustakaan (bus) selalu mengambil satu sekat penuh sekaligus. Kalau Anda menyimpan satu "jilid 4 buku" tepat di dalam satu sekat, petugas mengambilnya sekali jalan. Kalau jilid itu menyeberangi sekat (2 buku di sekat pertama, 2 di sekat kedua), petugas harus jalan dua kali, mengambil dua sekat, lalu memotong dan menyambung bukunya. Ada petugas yang mau melakukannya meskipun lebih lambat (x86, Cortex-M3 ke atas), dan ada petugas yang langsung menolak (Cortex-M0 dan banyak RISC lama).

Mengapa hardware peduli: empat lapisan alasan

1. Lebar bus dan desain unit load/store. Bus data 32-bit menghubungkan CPU ke memori dengan 32 jalur data. Memori SRAM biasanya diorganisasi per word 32-bit, dan dua bit terbawah alamat tidak memilih baris memori melainkan *byte lane* (byte keberapa di dalam word). Akses 32-bit yang aligned berarti satu transaksi bus. Akses yang menyeberang batas word berarti **dua** transaksi, ditambah rangkaian untuk menggeser dan menggabungkan byte. Di inti yang sangat kecil, rangkaian tambahan itu sengaja tidak dibuat untuk menghemat luas silikon dan daya. Itulah keputusan desain Cortex-M0/M0+: akses tak-aligned tidak didukung dan selalu memicu HardFault, tanpa opsi untuk dimatikan[19][20].

2. Cache line. Prosesor dengan cache tidak membaca memori per word, tetapi per **cache line**: 32 byte di Cortex-M7[34], 64 byte di x86 modern. Data yang menyeberangi batas line memaksa CPU mengakses dua line. Agner Fog melaporkan bahwa pada prosesor Intel sejak Sandy Bridge, pembacaan tak-aligned yang tetap di dalam satu line praktis tanpa penalti, dan sejak Haswell penalti lintas-line untuk *load* hilang, tetapi tidak untuk *store*[22]. Sebuah pengukuran di sepuluh CPU Intel menemukan penalti 0% untuk akses tak-aligned di dalam satu line, tetapi naik sampai sekitar 30–70% ketika akses memecah line L1[23].

3. Halaman memori virtual. Di sistem dengan MMU (Linux, Android), akses yang menyeberangi batas halaman 4 KB membutuhkan dua terjemahan alamat. Kalau halaman kedua belum dimuat, satu instruksi bisa memicu *page fault* di tengah-tengah akses, dan hardware harus mampu

membatalkan setengah akses yang sudah terjadi. Laporan di forum Intel menyebut tulisan yang menyeberangi halaman 4 KB pada Sandy Bridge memakan penalti lebih dari 100 siklus[63].

4. Atomisitas. Instruksi atomik (misalnya `LDREX` / `STREX` di ARM, `LOCK CMPXCHG` di x86) dirancang untuk mengunci satu lokasi. Di ARM, akses eksklusif ke alamat tak-aligned langsung gagal dengan fault. x86 masih mengizinkan operasi atomik yang terbelah di dua cache line (*split lock*), tetapi harganya sangat mahal karena dapat menghambat seluruh sistem, sehingga Linux menambahkan deteksi *split lock*, dan beberapa CPU Intel bisa memicu pengecualian `#AC` untuknya[24].

Peta perilaku per arsitektur

Arsitektur	Akses tak-aligned ke memori biasa	Biaya	Catatan
x86 / x86-64	Diizinkan untuk instruksi biasa	Hampir nol di dalam satu cache line; lebih mahal lintas line/halaman[2223]	Instruksi SSE tertentu (`movaps`) tetap mewajibkan alignment 16
Cortex-M0 / M0+ (ARMv6-M)	**Tidak**: HardFault[1920]	—	Compiler harus memecah akses jadi per byte jika alignment tidak diketahui
Cortex-M3/M4/M7/M33 (ARMv7-M, ARMv8-M)	Ya untuk `LDR` , `STR` , `LDRH` , `STRH` ; tidak untuk `LDM` , `STM` , `LDRD` , `STRD` , eksklusif[60]	Beberapa siklus tambahan (dipecah jadi beberapa transaksi bus)	Bit `UNALIGN_TRP` di register CCR bisa membuat semua akses tak-aligned menjadi fault[60]
AArch64 (Cortex-A, Apple M)	Ya untuk memori Normal	Kecil	Ke memori Device tetap fault
RISC-V	**Tergantung implementasi**: bisa didukung hardware, ditangani perangkat lunak lewat *trap*, atau fatal[25]	Dari nol sampai ratusan siklus (emulasi)	Ekstensi profil **Zicclsm** menandai dukungan; QEMU baru menambah emulasi "jujur" untuknya pada 2026[26]

Pelajaran dari tabel ini: **"berjalan di PC saya" tidak membuktikan apa pun tentang alignment.** Kode parsing yang diuji di laptop x86 bisa langsung *crash* di modul pembaca RFID berbasis Cortex-M0+.

Bagaimana compiler menjaga alignment: padding struct

Compiler menempatkan field-field struct **berurutan sesuai deklarasi** (C tidak mengizinkan compiler menyusun ulang), tetapi menyisipkan byte kosong (*padding*) agar setiap field memenuhi syarat alignment-nya. Alignment struct sendiri sama dengan alignment terbesar di antara anggotanya[18], dan ukurannya dibulatkan ke kelipatan alignment itu supaya elemen-elemen dalam array tetap aligned.

Contoh catatan hasil pembacaan tag di firmware pembaca (ABI Arm 32-bit):

```
struct baca_tag {          /* offset  ukuran */
    uint8_t  antena;       /*   0       1    */
                           /*   1..3    3    padding */
    uint32_t waktu_ms;     /*   4       4    */
    uint8_t  rssi;         /*   8       1    */
                           /*   9       1    padding */
    uint16_t pc;           /*  10       2    */
};                         /* sizeof = 12, alignof = 4 */
```

Empat dari dua belas byte (33%) terbuang untuk padding. Menyusun ulang field dari yang terbesar ke terkecil menghapus semuanya:

```
struct baca_tag_rapi {
    uint32_t waktu_ms;     /*   0  */
    uint16_t pc;           /*   4  */
    uint8_t  antena;       /*   6  */
    uint8_t  rssi;         /*   7  */
};                         /* sizeof = 8 */
```

Untuk satu catatan, selisih 4 byte tidak berarti. Untuk *buffer* 2.000 hasil baca di mikrokontroler dengan RAM 32 KB, selisihnya 8 KB, seperempat RAM.

> **CONTOH PERBEDAAN ABI YANG TERKENAL**
>
> `struct { char c; double d; }` berukuran **16 byte** di ABI Arm 32-bit (karena `double` aligned 8), tetapi **12 byte** di ABI System V untuk x86 32-bit (Linux i386), yang hanya mewajibkan `double` di dalam struct aligned 4. Artinya, struct yang sama punya tata letak berbeda di dua mesin 32-bit. Inilah alasan utama mengapa **menyalin struct mentah langsung ke kabel atau file** (`write(fd, &s, sizeof s)`) adalah praktik yang rapuh.

`packed`: mematikan padding, dan harganya

Kadang kita butuh struct yang tata letaknya persis sama dengan format byte di kabel, misalnya header frame dari chip pembaca RFID. GCC dan Clang menyediakan atribut `packed`, yang menghapus padding antar-anggota; menerapkannya ke struct sama dengan menerapkannya ke setiap anggota[16]. GCC juga menjelaskan bahwa atribut `aligned` hanya bisa **menaikkan** alignment, sedangkan untuk **menurunkannya** dipakai `packed`[16].

```
struct __attribute__((packed)) frame_hdr {
    uint8_t  sof;        /* 0 */
    uint16_t panjang;    /* 1..2  ← tak-aligned! */
    uint8_t  perintah;   /* 3 */
    uint32_t nomor;      /* 4..7 */
};                       /* sizeof = 8, alignof = 1 */
```

Ketika kode mengakses `hdr->panjang` **langsung lewat struct**, compiler tahu field itu mungkin tak-aligned, sehingga di Cortex-M0 ia menghasilkan dua `LDRB` dan penggabungan. Aman, tetapi lebih lambat dan lebih besar. Masalah muncul ketika alamat field itu "dicuci" menjadi pointer biasa:

```
uint16_t *p = &hdr->panjang;   /* informasi "mungkin tak-aligned" HILANG */
uint16_t n = *p;               /* compiler memakai LDRH → HardFault di M0 */
```

Tipe `uint16_t *` membawa janji alignment 2. Compiler mempercayai janji itu. GCC sejak versi 9 memberi peringatan `-Waddress-of-packed-member` untuk pola ini. Diskusi di forum Arm dan Launchpad mencatat kasus nyata: struct packed yang diakses lewat pointer acak di Cortex-M0 dengan `-Os` menghasilkan HardFault, dan solusinya adalah `memcpy()`[2119].

Cara yang benar membaca data tak-aligned

Ada dua pola yang portabel dan, dengan compiler modern, sama sekali tidak lambat:

```
#include <string.h>

/* Pola 1: memcpy — compiler tahu alignment sumber hanya 1 */
static inline uint32_t baca_u32_ne(const uint8_t *p) {
    uint32_t v;
    memcpy(&v, p, sizeof v);   /* urutan byte = urutan mesin (little-endian di Cortex-M)
*/
    return v;
}

/* Pola 2: rakit dari byte — sekaligus menentukan endianness secara eksplisit */
static inline uint32_t baca_u32_be(const uint8_t *p) {
    return ((uint32_t)p[0] << 24) | ((uint32_t)p[1] << 16) |
           ((uint32_t)p[2] << 8)  |  (uint32_t)p[3];
}
```

`memcpy` dengan ukuran konstan kecil tidak benar-benar memanggil fungsi pustaka. Compiler mengenalinya sebagai *built-in*: di Cortex-M4 ia menjadi satu `LDR` (karena M4 mendukung `LDR` tak-aligned), di Cortex-M0 menjadi empat `LDRB`, dan di x86 menjadi satu `mov`. Ini contoh terbaik dari "kontrak yang jujur": programmer menyatakan apa yang sebenarnya ia ketahui (alamat bisa tak-aligned), lalu compiler memilih instruksi terbaik untuk chip yang dituju. Pola 2 bahkan sering dikenali dan diganti dengan `LDR` + `REV` (instruksi pembalik urutan byte) di Cortex-M3 ke atas.

Menaikkan alignment: kapan dan mengapa

Kebalikan dari `packed`, kadang kita perlu alignment **lebih besar** daripada bawaan:

Kebutuhan	Alignment	Cara
Buffer DMA di Cortex-M7 dengan cache	32 byte, ukuran kelipatan 32[34]	`alignas(32) uint8_t rx_buf[256];`
Tabel vektor yang dipindahkan ke RAM (`VTOR`)	Minimal 128 byte, tergantung jumlah vektor	`ALIGN(...)` di linker script
Stack di batas fungsi publik (AAPCS)	8 byte[17]	Diatur linker script dan kode startup
Region proteksi MPU (ARMv7-M)	Sebesar ukuran region (pangkat dua)	Alignment di linker script
Data SIMD (SSE/NEON/Helium)	16 byte atau lebih	`alignas(16)`
Menghindari *false sharing* antar-core	64 byte (x86)	`alignas(64)` per variabel yang dipakai thread berbeda

Di C11 cukup menulis `alignas(N)` (dari `<stdalign.h>`)[14]; di GCC/Clang lama, `__attribute__((aligned(N)))` . Untuk memori dinamis ada `aligned_alloc()` (C11).

DMA, cache, dan alignment: jebakan terbesar di Cortex-M7

Cortex-M7 punya cache data. Ketika CPU menulis ke buffer, data mungkin baru masuk cache dan belum sampai ke SRAM. Ketika DMA menulis ke SRAM, cache CPU mungkin masih menyimpan versi lama. Kedua pihak "melihat" isi memori yang berbeda, karena Cortex-M7 tidak menjaga koherensi cache dengan DMA secara otomatis[34]. Aturannya[3435]:

- **Sebelum DMA mengirim** (TX, misalnya perintah ke chip RFID lewat SPI): *clean* cache untuk buffer itu (`SCB_CleanDCache_by_Addr`), supaya isi cache tertulis ke SRAM.
- **Setelah DMA menerima** (RX, misalnya respons tag): *invalidate* cache untuk buffer itu (`SCB_InvalidateDCache_by_Addr`) sebelum CPU membacanya.
- **Alamat awal buffer harus aligned 32 byte, dan ukurannya kelipatan 32.** Fungsi-fungsi CMSIS bekerja per line utuh; alamat yang tak-aligned dibulatkan ke bawah diam-diam[36]. Jika buffer berbagi satu line dengan variabel lain, *invalidate* bisa **menghapus nilai terbaru variabel tetangga** itu[35].
- **Alternatif:** tandai wilayah buffer DMA sebagai *non-cacheable* lewat MPU. Banyak contoh resmi ST memilih cara ini[34].

MENGAPA BUG INI BEGITU SULIT DILACAK

Bug koherensi cache tidak muncul di debugger ketika Anda menghentikan program, karena debugger sering membaca memori lewat jalur yang berbeda dari CPU. Bug ini muncul acak, tergantung kapan line cache kebetulan dikeluarkan, dan sering hilang ketika Anda menambahkan `printf` (yang mengubah isi cache). Gejala tipikal di pembaca RFID: sesekali EPC yang terbaca berisi data dari pembacaan **sebelumnya**.

Daftar periksa alignment untuk firmware

1. Jangan pernah melakukan *cast* `uint8_t *` ke `uint16_t *` / `uint32_t *` untuk membaca data dari buffer komunikasi. Gunakan `memcpy` atau perakitan byte.
2. Jangan mengambil alamat anggota struct `packed`. Aktifkan `-Wall -Wextra -Waddress-of-packed-member -Wcast-align=strict`.
3. Susun field struct dari yang terbesar ke terkecil kalau tata letak tidak terikat format eksternal.
4. Pakai `_Static_assert(sizeof(struct x) == 8, "...")` dan `offsetof` untuk mengunci tata letak yang penting.
5. Buffer DMA di chip ber-cache: `alignas(32)`, ukuran kelipatan 32, atau letakkan di wilayah non-cacheable.
6. Kompilasi dengan `-mcpu` yang benar. Kode untuk `cortex-m4` yang dijalankan di M0+ akan memakai instruksi dan asumsi alignment yang tidak dimiliki M0+.
7. Uji di hardware target, bukan hanya di PC.

Ketika Compiler dan Hardware Salah Paham

`volatile`: "jangan berasumsi tentang memori ini"

Kata kunci `volatile` dirancang untuk objek khusus, seperti register perangkat, yang isinya bisa berubah sendiri[9]. Ia memberi tahu compiler bahwa **setiap** akses ke objek itu adalah perilaku yang bisa diamati: tidak boleh dihapus, digabung, diduplikasi, atau disusun ulang terhadap akses `volatile` lain. Karena itu pola standar akses register adalah pointer ke tipe `volatile`[11]:

```
#define UART_SR  (*(volatile uint32_t *)0x40013800u)
while ((UART_SR & RXNE) == 0) { }   /* sekarang register dibaca di setiap putaran */
```

Kerangka CMSIS dari Arm memakai pola yang sama dalam bentuk lebih rapi: sebuah struct berisi anggota `volatile` (lewat makro `__IO`, `__I`, `__O`) yang dipetakan ke alamat dasar periferal, misalnya `USART1->SR`. Struct dipakai untuk tata letak word-word register, sedangkan field di dalam setiap register didefinisikan sebagai *mask* dengan `#define`[41].

Namun `volatile` sering dianggap lebih sakti dari kenyataannya. John Regehr mendaftar "sembilan cara merusak kode sistem dengan `volatile`"[10]. Yang terpenting:

- `volatile` **tidak membuat operasi atomik.** `counter++` pada variabel `volatile` tetap tiga instruksi (baca, tambah, tulis). Interupsi di tengahnya bisa menghilangkan satu penambahan.
- `volatile` **tidak mengurutkan akses non-**`volatile`**.** Compiler masih boleh memindahkan tulisan ke buffer biasa melewati tulisan ke register `volatile` yang memulai DMA. Akibatnya DMA bisa mulai sebelum buffer terisi.
- `volatile` **tidak mengatur urutan di tingkat hardware.** Ia hanya berbicara kepada compiler. Apa yang dilakukan bus, *write buffer*, atau cache adalah urusan lain.
- **Dokumentasi GNU sendiri mengingatkan** bahwa standar C tidak merinci perilaku persis untuk kasus seperti ini, sehingga detailnya bisa berbeda antar-platform[9].

Barrier: dua jenis pagar

Untuk mengatasi dua kelemahan terakhir, ada dua jenis "pagar" yang berbeda tingkatannya:

Compiler barrier melarang compiler memindahkan akses memori melewati satu titik. Di GCC/Clang bentuknya `asm volatile("" ::: "memory");`. Wiki OSDev menyarankan memakai barrier eksplisit seperti ini daripada mengandalkan `volatile` sebagai pengganti barrier[11]. Instruksi ini tidak menghasilkan satu instruksi mesin pun. Ia murni pesan kepada optimizer.

Hardware barrier melarang **prosesor** dan sistem memori menyusun ulang akses. Di ARM ada tiga[3233]:

Instruksi	Nama	Arti
`DMB`	*Data Memory Barrier*	Akses memori sebelum `DMB` terlihat oleh pihak lain sebelum akses sesudahnya
`DSB`	*Data Synchronization Barrier*	Seperti `DMB`, ditambah menunggu sampai semua akses yang tertunda benar-benar selesai
`ISB`	*Instruction Synchronization Barrier*	Mengosongkan pipeline, sehingga instruksi sesudahnya diambil ulang dengan konteks baru (misalnya setelah mengubah `VTOR` atau MPU)

CMSIS menyediakannya sebagai fungsi intrinsik `__DMB()`, `__DSB()`, dan `__ISB()`, yang juga berlaku sebagai compiler barrier. Arm menerbitkan panduan khusus, *Application Note 321*, tentang kapan barrier dibutuhkan di Cortex-M[32]. Dalam praktik, inti Cortex-M sederhana (M0, M3, M4) jarang menyusun ulang akses memori, sehingga banyak kode berjalan benar tanpa barrier. Itu sebabnya ada pendapat di forum Arm bahwa barrier di inti-inti ini nyaris tidak pernah diperlukan[33]. Tetapi Cortex-M7, dengan pipeline superskalar dan cache, **berbeda**, dan kode yang ingin portabel antar-inti sebaiknya mengikuti panduan AN321. Contoh klasik: setelah menonaktifkan interupsi atau mengubah konfigurasi MPU, tambahkan `__DSB(); __ISB();`.

Strict aliasing: compiler percaya pada tipe

Aturan *strict aliasing* menyatakan bahwa objek bertipe `T` hanya boleh diakses lewat lvalue bertipe `T` (atau varian yang kompatibel, atau `char`). Melanggarnya, misalnya membaca `float` lewat `uint32_t *`, adalah *undefined behavior*[1213]. Mengapa aturan ini ada? Karena ia memberi optimizer informasi berharga: tulisan lewat `uint16_t *` **pasti** tidak mengubah `uint32_t` mana pun, sehingga nilai `uint32_t` yang sudah ada di register tidak perlu dibaca ulang.

```
uint32_t baca_epc_word(uint8_t *buf) {
    uint32_t *w = (uint32_t *)buf;   /* melanggar aliasing (dan mungkin alignment) */
    buf[0] = 0x30;                   /* compiler boleh menganggap ini tak mengubah *w */
    return *w;
}
```

Di GCC, analisis ini aktif secara bawaan pada `-O2` ke atas (`-fstrict-aliasing`), dan bisa dimatikan dengan `-fno-strict-aliasing`. Kernel Linux, misalnya, sengaja dikompilasi dengan opsi itu. Solusi yang sah dan efisien kembali sama: `memcpy`[1213]. Pengecualian penting: tipe karakter (`char`, `unsigned char`, `uint8_t` di praktiknya) **boleh** mengakses objek apa pun, karena itulah cara kerja `memcpy` itu sendiri.

Endianness: urutan byte di dalam word

Bilangan 32-bit `0x30742578` disimpan di memori little-endian (Cortex-M, x86, RISC-V) sebagai byte `78 25 74 30` dari alamat terkecil, dan di memori big-endian sebagai `30 74 25 78`. CPU tidak "salah" dalam kedua kasus; yang penting adalah **kesepakatan** antara pengirim dan penerima. Protokol jaringan dan banyak protokol RFID mengirimkan bit atau byte paling signifikan lebih dulu ("urutan jaringan"). Ketika firmware little-endian menerima angka big-endian dan membacanya lewat `memcpy` ke `uint32_t`, hasilnya terbalik. Solusinya:

- rakit byte secara eksplisit seperti `baca_u32_be()` di atas, atau
- gunakan `__builtin_bswap32()` / `__REV()` (CMSIS), yang menjadi satu instruksi `REV` di Cortex-M3 ke atas.

Bit-field: nyaman tetapi tidak portabel

C mengizinkan struct berisi field selebar beberapa bit:

```
struct pc_word { uint16_t panjang:5; uint16_t umi:1; uint16_t xi:1; uint16_t t:1;
uint16_t afi:8; };
```

Kelihatannya ideal untuk memodelkan register atau word protokol. Masalahnya, standar C membiarkan **urutan alokasi bit di dalam satu unit** (dari bit tinggi ke rendah atau sebaliknya) sebagai *implementation-defined*, dan alignment unit penyimpanannya tidak ditentukan[41][62]. Dua compiler bisa menempatkan `panjang` di bit 0–4 atau di bit 11–15. Untuk register hardware ada risiko tambahan: menulis satu bit-field menghasilkan *read-modify-write* atas seluruh word, yang bisa tak sengaja mengubah bit lain atau membersihkan flag *write-1-to-clear*[41]. Karena itulah CMSIS dan sebagian besar driver profesional memakai *mask* dan *shift* eksplisit. Sebagian praktisi berpendapat risiko ini bisa diterima selama seluruh proyek memakai satu compiler yang perilakunya diuji. Ini bagian dari perdebatan yang dibahas belakangan.

Studi Kasus RFID: Dari Udara ke Server

Tiga frekuensi, tiga dunia

"RFID" sebenarnya payung untuk beberapa teknologi yang sangat berbeda[58]:

Pita	Frekuensi	Standar utama	Jarak baca tipikal	Contoh penggunaan
LF	125–134 kHz	ISO 11784/11785, berbagai format kartu proksimitas	Beberapa sentimeter	Kartu akses lama, identifikasi hewan
HF / NFC	13,56 MHz	ISO/IEC 14443 (kartu, MIFARE), ISO/IEC 15693 (*vicinity*), NFC	Sentimeter sampai ±1 m	Kartu transportasi, perpustakaan, ponsel
UHF (RAIN)	860–960 MHz	GS1 EPC UHF Gen2 = ISO/IEC 18000-63	Beberapa meter	Ritel, logistik, gudang, inventaris

Bab ini berfokus pada **UHF Gen2** karena paling kaya contoh soal bit, word, dan alignment, dengan beberapa catatan tentang HF.

Sisi tag: chip tanpa CPU, tanpa compiler

Chip tag UHF pasif tidak punya baterai. Antena tag menangkap energi gelombang radio dari pembaca, lalu rangkaian penyearah di chip mengubahnya menjadi tegangan untuk menghidupkan logika. Gelombang pembawa yang sama dipakai untuk dua hal sekaligus: dipanen sebagai daya dan dipantulkan kembali sebagai jawaban[47]. Demodulator memulihkan data dari pembaca, lalu menyerahkannya ke sebuah **mesin keadaan** (*state machine*) yang memutuskan respons[47]. Tag menjawab dengan teknik **backscatter**: mengubah impedansi antena di antara dua keadaan sehingga jumlah sinyal yang dipantulkan berubah-ubah sesuai bit data.

Di sinilah ada pelajaran menarik bagi topik kita. **Tidak ada program, tidak ada CPU, dan tidak ada compiler perangkat lunak di dalam chip tag.** Seluruh protokol Gen2 (menerima `Query`, menghitung *slot*, memilih bilangan acak, membaca memori) diwujudkan sebagai rangkaian logika khusus, karena CPU umum terlalu boros daya untuk hidup dari energi yang dipanen dari jarak beberapa meter. Tetapi rangkaian itu sendiri pun lahir dari sebuah "compiler": insinyur menulis perilaku chip dalam bahasa deskripsi hardware (Verilog atau VHDL), lalu perangkat **sintesis logika** menerjemahkannya menjadi jaringan gerbang AND/OR/flip-flop, mengoptimalkannya, dan memetakannya ke pustaka sel milik pabrik semikonduktor. Prinsipnya sama dengan compiler C (front-end, representasi antara, optimisasi, pemetaan ke target), hanya saja targetnya transistor, bukan instruksi.

Memori tag: empat bank, word 16-bit

Memori tag Gen2 dibagi menjadi empat **bank**, dengan kode 2-bit yang dipakai dalam perintah[42][43]:

Kode	Bank	Isi	Catatan
`00`	Reserved	*Kill password* (32 bit) dan *access password* (32 bit)	Bisa dikunci sehingga tidak terbaca[42]
`01`	EPC	StoredCRC, *Protocol Control* (PC), lalu EPC	EPC minimal 96 bit; PC menyatakan panjang EPC[42]
`10`	TID	Identitas chip dari pabrik (pembuat, model, sering nomor seri)	Ditulis pabrik, tidak bisa diubah[42]
`11`	User	Data aplikasi	Opsional; ukurannya sangat bervariasi antar-chip[42]

Yang penting untuk pembahasan alignment: **memori Gen2 dialamatkan per *word* 16-bit, bukan per byte.** Perintah `Read` menyebutkan bank, alamat word awal (`WordPtr`), dan jumlah word (`WordCount`)[44]. Di bank EPC, word 0 berisi StoredCRC, word 1 berisi PC, dan EPC dimulai di word 2 (alamat bit `0x20`). Akibatnya, "alignment" juga berlaku di udara: Anda tidak bisa membaca atau menulis setengah word, dan EPC 96 bit selalu menempati tepat enam word.

Memori ini adalah memori non-volatil (NVM) yang lambat ditulis dan punya umur tulis terbatas. Chip NXP UCODE 9, misalnya, memiliki EPC 96 bit dan dijamin minimal 100.000 siklus tulis[45]; variannya, UCODE 9xm, memungkinkan pembagian memori yang bisa dikonfigurasi pengguna antara EPC dan User (misalnya EPC 128 bit/User 752 bit sampai EPC 496 bit/User 384 bit)[46].

Antikolisi: algoritma Q

Bagaimana pembaca membedakan ratusan tag yang menjawab bersamaan? Gen2 memakai varian **slotted ALOHA**. Pembaca mengirim `Query` yang membawa parameter **Q**. Setiap tag memilih angka acak antara 0 dan 2^Q − 1 sebagai penghitung slot. Tag yang penghitungnya nol menjawab dengan bilangan acak 16-bit (**RN16**); pembaca membalas `ACK`, dan tag mengirim PC + EPC + CRC-16. Perintah `QueryRep` mengurangi penghitung semua tag, sehingga tag berikutnya mendapat giliran[48]. Jika terlalu banyak tabrakan, pembaca menaikkan Q (lebih banyak slot); jika terlalu banyak slot kosong, menurunkannya. Spesifikasi menyediakan contoh algoritma dengan nilai awal sekitar 4 dan konstanta penyesuaian kecil, tetapi pembuat pembaca bebas memakai strategi sendiri[44][48]. Bagi programmer firmware, ini berarti satu perintah "inventaris" di level aplikasi sebenarnya memicu ratusan transaksi kecil di udara, semuanya dikerjakan oleh chip pembaca.

Sisi pembaca: lapisan di dalam lapisan

Pembaca UHF modern adalah tumpukan komputer kecil:

```
[Aplikasi/server]  ←TCP/IP, LLRP atau REST/MQTT→  [Prosesor host di pembaca: Linux SoC atau MCU]
                                                          │  SPI / UART
                                                          ▼
                                  [Chip pembaca RFID: MCU internal + MAC + modem + radio]
                                                          │  RF 860–960 MHz
                                                          ▼
                                                [Antena] ~~~~~~ [Tag pasif]
```

Chip pembaca seperti **Impinj E710** menggabungkan transceiver radio, *front-end* RF, modem yang bisa dikonfigurasi, pembatalan *self-jammer*, manajemen daya, sebuah **MCU dengan MAC** (lapisan kendali protokol), dan **antarmuka host SPI**[49]. Generasi sebelumnya, Impinj Indy R2000, berkomunikasi lewat UART dan sudah memasuki proses akhir masa produksi sejak September 2022[50]; Impinj menyebut E710 memiliki sensitivitas hingga 9 dB lebih baik dibanding R2000[51]. Di sisi HF/NFC, chip seperti **ST25R3916B** dari STMicroelectronics menyediakan FIFO 512 byte dan SPI hingga 10 Mbit/s[52].

Perhatikan apa artinya: **di dalam chip pembaca itu sendiri ada firmware yang dikompilasi**, berjalan di MCU internal, dan tidak pernah Anda lihat. Firmware Anda di MCU host berbicara dengan firmware itu lewat SPI, persis seperti CPU berbicara dengan register periferal. Dari sudut pandang MCU host, chip pembaca adalah "periferal eksternal"; dari sudut pandang server, seluruh pembaca adalah "periferal jaringan". Abstraksi yang sama berulang di setiap lapisan.

Mengikuti satu EPC dari udara ke server

Mari ikuti satu pembacaan tag, dan tandai di mana compiler dan hardware bisa salah paham.

Langkah 1: tag menjawab. Bit-bit PC dan EPC dikirim lewat udara, bit paling signifikan lebih dulu, dilindungi CRC-16. Chip pembaca memeriksa CRC dan menyimpan hasil di buffer internalnya.

Langkah 2: chip pembaca memberi tahu host. Chip menarik pin IRQ. MCU host masuk ke *interrupt handler*. Di sini muncul jebakan pertama: forum ST mengingatkan bahwa jika ada perangkat lain di bus SPI yang sama, komunikasi SPI sebaiknya tidak dilakukan di dalam konteks interupsi. Pola yang umum adalah handler hanya membangunkan sebuah *task* yang kemudian membaca chip[53]. Errata ST juga mencatat bahwa pada revisi silikon tertentu, pola bit tertentu saat

membaca status interupsi bisa membuat flag tidak valid[59], pengingat bahwa hardware pun punya bug, dan driver yang baik menyertakan *workaround*-nya.

```
static volatile bool rfid_irq_flag;          /* diubah ISR, dibaca task: wajib volatile
*/

void EXTI4_IRQHandler(void) {
    EXTI->PR = EXTI_PR_PR4;                  /* bersihkan flag (write-1-to-clear) */
    rfid_irq_flag = true;                    /* atau: beri sinyal ke task RTOS */
}
```

Langkah 3: DMA menerima respons lewat SPI. Firmware memulai transfer DMA dari register data SPI ke buffer RAM. Di Cortex-M7, buffer ini harus `alignas(32)` dan cache-nya di-*invalidate* setelah transfer selesai[34]; kalau tidak, CPU mungkin membaca respons tag **sebelumnya**.

```
alignas(32) static uint8_t rx_buf[64];       /* 64 = kelipatan 32 */

void setelah_dma_selesai(void) {
    SCB_InvalidateDCache_by_Addr((uint32_t *)rx_buf, sizeof rx_buf);
    proses_frame(rx_buf, panjang_diterima);
}
```

Langkah 4: firmware mem-*parsing* frame. Misalkan (sebagai ilustrasi, mirip format banyak modul UHF serial) frame respons berbentuk:

Offset	Ukuran	Isi
0	1	Penanda awal `0xBB`
1	1	Tipe frame
2	1	Kode perintah
3	2	Panjang payload (big-endian)
5	1	RSSI
6	2	PC word (big-endian)
8	12	EPC (96 bit)
20	2	CRC-16 tag
22	1	Checksum frame
23	1	Penanda akhir `0x7E`

Perhatikan bahwa field 16-bit "panjang" berada di offset 3, **ganjil**. Kode berikut terlihat wajar, dan bahkan berjalan di laptop:

```
/* SALAH: tiga pelanggaran sekaligus */
uint16_t panjang = *(uint16_t *)(rx_buf + 3);  /* 1) tak-aligned  2) aliasing  3)
endianness salah */
```

Di Cortex-M0+ ini menghasilkan HardFault. Di Cortex-M4 ia "jalan" tetapi menghasilkan nilai dengan byte terbalik, karena data big-endian dibaca oleh CPU little-endian. Di compiler dengan optimisasi agresif, pelanggaran aliasing bisa membuat nilai yang dibaca tidak sinkron dengan isi buffer. Versi yang benar sederhana dan sama cepatnya:

```
static inline uint16_t be16(const uint8_t *p) { return (uint16_t)((p[0] << 8) | p[1]); }

bool proses_frame(const uint8_t *f, size_t n) {
    if (n < 24 || f[0] != 0xBB || f[n - 1] != 0x7E) return false;
    uint16_t panjang = be16(f + 3);
    int8_t   rssi    = (int8_t)f[5];
    uint16_t pc      = be16(f + 6);
    unsigned epc_words = pc >> 11;          /* 5 bit teratas PC = panjang EPC dalam
word */
    const uint8_t *epc = f + 8;             /* tetap sebagai byte, tidak di-cast */
    /* ... validasi checksum dan CRC, lalu simpan ... */
    return epc_words == 6;
}
```

Kalau ingin memakai struct untuk kerapian, gunakan struct **biasa** (bukan packed) sebagai representasi di dalam program, lalu isi field-fieldnya dari buffer byte dengan fungsi seperti `be16()`. Pisahkan dengan tegas **format di kabel** (deretan byte, urutan jaringan) dari **format di memori** (struct, aligned, urutan mesin).

Langkah 5: menerjemahkan EPC. EPC 96 bit biasanya berisi data GS1 yang dikodekan sesuai *Tag Data Standard*. Byte pertama adalah **header**; nilai `0x30` menandakan skema **SGTIN-96**, yaitu GTIN (nomor barang) berseri yang dikodekan dalam 96 bit[56]. Strukturnya: header 8 bit, *filter* 3 bit, *partition* 3 bit, *company prefix* + *item reference* total 44 bit (pembagiannya ditentukan nilai partition), dan nomor seri 38 bit[57]. Field-field ini **tidak sejajar dengan batas byte**, jadi perlu ekstraksi per bit:

```
static uint64_t ambil_bit(const uint8_t *b, unsigned awal, unsigned n) {
    uint64_t v = 0;
    for (unsigned i = 0; i < n; i++) {
        unsigned pos = awal + i;                      /* bit 0 = MSB byte 0 */
        v = (v << 1) | ((b[pos >> 3] >> (7 - (pos & 7))) & 1u);
    }
    return v;
}

/* EPC contoh: 30 74 25 7B F7 19 4E 40 00 00 1A 85 */
header   = ambil_bit(epc,  0,  8);   /* 0x30 → SGTIN-96          */
filter   = ambil_bit(epc,  8,  3);   /* 3                        */
partisi  = ambil_bit(epc, 11,  3);   /* 5 → prefix 24 bit, item 20 bit */
prefix   = ambil_bit(epc, 14, 24);   /* 614141                   */
item     = ambil_bit(epc, 38, 20);   /* 812345                   */
serial   = ambil_bit(epc, 58, 38);   /* 6789                     */
```

EPC contoh di atas, yang sering dipakai dalam dokumentasi GS1, menghasilkan *company prefix* 0614141, *item reference* 812345, dan nomor seri 6789. Kode ini sengaja ditulis per bit agar jelas dan bebas dari masalah alignment maupun endianness. Versi yang dioptimalkan bisa memuat 96 bit ke dua variabel 64-bit lalu memakai *shift* dan *mask*, tetapi pemuatan itu tetap harus dilakukan dengan perakitan byte big-endian, bukan *cast*.

Langkah 6: mengirim ke server. Pembaca kelas industri sering memakai **LLRP** (*Low Level Reader Protocol*), standar GS1 untuk komunikasi pembaca dan klien, kini dalam Rilis 2.0 (2021)[54]. LLRP berjalan di atas TCP dengan port IANA 5084, dan pesannya dikodekan biner dengan header tetap berisi tipe, panjang, dan ID pesan[55]. Parameter di dalamnya berbentuk TLV (tipe–panjang–nilai) dengan header berisi bit cadangan, tipe 10-bit, dan panjang 16-bit[55]; bilangan dikirim dengan urutan byte jaringan (paling signifikan lebih dulu). Sekali lagi berlaku aturan yang sama: **serialisasi field demi field**, jangan mengirim struct mentah, karena padding dan endianness struct di memori tidak sama dengan format di kabel.

RINGKASAN: TITIK RAWAN DI PEMBACA RFID

Titik	Kontrak yang bisa dilanggar	Pencegahan
Flag dari ISR	Compiler menyimpan flag di register	`volatile` (atau atomik C11)
Penulisan register chip lewat SPI	Urutan akses buffer vs register DMA	Barrier compiler/ `__DSB()` sebelum memulai DMA
Buffer DMA di Cortex-M7	Koherensi cache	`alignas(32)`, ukuran kelipatan 32, *clean/invalidate*, atau MPU non-cacheable
Parsing frame	Alignment, aliasing, endianness	`be16()` / `memcpy`, jangan *cast* pointer
Struct protokol	Padding beda antar-ABI	Serialisasi eksplisit; `_Static_assert`
Memori tag	Alamat dan panjang dalam word 16-bit	Hitung `WordPtr` / `WordCount`, bukan byte
Ganti MCU (M4 → M0+)	Akses tak-aligned diizinkan → fault	`-mcpu` benar, uji di target

Data & Angka Kunci

Perbandingan arsitektur dan alignment

Parameter	Cortex-M0/M0+	Cortex-M3/M4	Cortex-M7	x86-64 modern
Akses tak-aligned (LDR/STR biasa)	Fault[1920]	Didukung, lebih lambat[60]	Didukung, lebih lambat	Didukung
Akses tak-aligned untuk banyak-register/eksklusif	Fault	Fault[60]	Fault	Atomik lintas line = *split lock*, sangat mahal[24]
Ukuran cache line	Tanpa cache	Tanpa cache	32 byte[34]	64 byte[22]
Penalti tak-aligned dalam satu line	—	Beberapa siklus	Beberapa siklus	≈0%[23]
Penalti tak-aligned lintas line	—	—	Dua line diakses	±30–70% (pengukuran 10 CPU Intel)[23]
Instruksi	Thumb (sebagian besar 16-bit)	Thumb-2 (16/32-bit)[31]	Thumb-2	x86 (1–15 byte)

Aturan alignment ABI 32-bit (ringkas)

Tipe	Ukuran	Alignment AAPCS (Arm 32-bit)[17]	Alignment i386 System V (di dalam struct)
`char`, `uint8_t`	1	1	1
`uint16_t`	2	2	2
`uint32_t`, `float`, pointer	4	4	4
`uint64_t`, `double`	8	**8**	**4**
Stack di batas fungsi publik	—	8[17]	4 (i386 lama), 16 (Linux modern)

Angka-angka RFID

Hal	Nilai	Sumber
Bank memori tag Gen2	4 (Reserved, EPC, TID, User)	42
Satuan alamat memori Gen2	Word 16-bit	44
Kill/access password	Masing-masing 32 bit	42
EPC minimum	96 bit	42
UCODE 9: EPC / siklus tulis	96 bit / ≥100.000	45
Header SGTIN-96	`0x30` (48 desimal)	56
SGTIN-96: header/filter/partition/prefix+item/serial	8/3/3/44/38 bit	57
Port TCP LLRP	5084	55
Impinj E710 vs R2000	Sensitivitas hingga 9 dB lebih baik	51
ST25R3916B	FIFO 512 byte, SPI hingga 10 Mbit/s	52

Perdebatan & Perspektif

Optimisasi berbasis *undefined behavior*: fitur atau jebakan?

Kubu pembuat compiler berpendapat bahwa memanfaatkan UB (misalnya menganggap pointer selalu aligned, atau *signed overflow* tidak pernah terjadi) adalah sumber utama kecepatan kode modern, dan bahwa standar memberi hak itu dengan jelas. Kubu programmer sistem berpendapat bahwa C dipakai justru karena dekat dengan hardware, sehingga compiler yang "menghukum" kode yang dulu berjalan selama puluhan tahun merusak kepercayaan. John Regehr dan kawan-kawan pernah mengusulkan dialek "Friendly C" yang mengubah banyak UB menjadi perilaku yang terdefinisi, dan proyek besar seperti kernel Linux memilih jalan tengah dengan opsi seperti `-fno-strict-aliasing`. Kedua sisi punya argumen kuat; yang tidak bisa diperdebatkan adalah bahwa **programmer harus tahu di sisi kontrak mana kodenya berdiri**.

Apakah `volatile` sudah cukup?

Sebagian pengembang embedded memakai `volatile` sebagai obat untuk semua masalah konkurensi. Kritikus seperti Regehr menunjukkan bahwa `volatile` tidak memberi atomisitas maupun pengurutan terhadap akses non-`volatile`[10]. Komite C++ bahkan menandai beberapa pemakaian `volatile` sebagai *deprecated* di C++20 (misalnya operator majemuk seperti `+=` pada objek `volatile`), lalu sebagian dicabut kembali di C++23 setelah keberatan dari komunitas embedded yang memakainya untuk register. Perdebatan ini memperlihatkan ketegangan abadi: `volatile` adalah alat yang tumpul tetapi universal, sementara atomik C11/C++11 dan barrier lebih tepat tetapi lebih rumit.

Bit-field dan struct packed untuk register dan protokol

Satu kubu (termasuk gaya CMSIS) menolak bit-field untuk register karena tata letaknya *implementation-defined* dan bisa memicu *read-modify-write* tak terduga[41]. Kubu lain menganggapnya dapat diterima selama toolchain tetap dan diuji, karena kodenya lebih mudah dibaca[62]. Hal serupa berlaku untuk struct `packed`: praktis untuk memetakan format kabel, tetapi rentan di inti tanpa dukungan akses tak-aligned[1921].

C, C++, atau Rust untuk firmware?

Rust menjanjikan keamanan memori tanpa *garbage collector*, dan ekosistem embedded-nya (`cortex-m-rt`, `embedded-hal`, `svd2rust`) sudah matang untuk banyak keluarga MCU[3738]. Rust tidak menghapus masalah alignment atau MMIO. Ia **memindahkannya** ke API yang lebih eksplisit:

`read_volatile` / `write_volatile`, `read_unaligned`, dan larangan membuat referensi ke field struct packed. Pendukung C menunjuk ke ekosistem vendor, sertifikasi keselamatan (MISRA C), dan tenaga kerja yang sudah ada. Pendukung Rust menunjuk ke kelas bug yang hilang sama sekali. Untuk proyek RFID baru dengan tim kecil, keduanya sah; untuk perangkat yang memerlukan driver vendor (misalnya pustaka RFAL dari ST atau SDK chip pembaca), C masih jalur termudah.

Assembly tulisan tangan

Dahulu, kode kritis ditulis langsung dalam assembly. Sejak FORTRAN membuktikan kode mesin buatan compiler bisa menyaingi manusia[4], ruang untuk assembly tulisan tangan terus menyempit. Hari ini assembly masih dipakai untuk kode startup tertentu, *context switch* RTOS, dan rutinitas DSP yang sangat spesifik. Selebihnya, compiler biasanya menang, terutama dengan *intrinsics* yang memberi akses ke instruksi khusus tanpa meninggalkan C.

Kondisi Terkini & Prospek

Keadaan per November 2026.

RISC-V dan pertanyaan alignment yang belum selesai. RISC-V terus menyebar di mikrokontroler dan SoC IoT. Spesifikasinya mengizinkan implementasi menangani akses tak-aligned dengan cara berbeda-beda, dari hardware penuh sampai *trap* fatal[25]. Profil-profil RISC-V memakai ekstensi **Zicclsm** untuk menyatakan dukungan akses tak-aligned, dan pada pertengahan 2026 pengembang SiFive mengajukan serangkaian patch agar emulator QEMU bisa memunculkan pengecualian tak-aligned dengan benar ketika Zicclsm dimatikan[26]. Bagi pengembang, implikasinya tetap sama: jangan berasumsi, gunakan `memcpy`.

LLVM sebagai fondasi bersama. LLVM kini menjadi dasar Clang, Rust, Swift, dan toolchain komersial Arm. Ini berarti optimisasi (dan perilaku terkait alignment serta aliasing) makin seragam di berbagai bahasa, dan peningkatan di satu tempat dinikmati semua.

Inti dengan cache dan vektor makin umum di MCU. Cortex-M7, M55, dan M85 membawa cache dan instruksi vektor ke dunia mikrokontroler. Masalah yang dulu hanya dikenal di dunia PC dan ponsel, seperti koherensi cache dengan DMA dan alignment untuk SIMD, kini harus dipahami juga oleh insinyur firmware[3435].

RFID: chip pembaca makin terintegrasi. Generasi chip pembaca seperti Impinj E-series memindahkan lebih banyak fungsi (MAC, pembatalan *self-jammer*) ke dalam chip dengan antarmuka SPI sederhana[49], sementara generasi lama seperti R2000 sudah memasuki akhir masa produksi[50]. Bagi pengembang, firmware host makin tipis, tetapi kebutuhan memahami DMA, interupsi, dan parsing biner tidak berkurang. Di sisi tag, chip baru seperti UCODE 9xm memberi fleksibilitas tata letak memori[46], sehingga perangkat lunak pembaca harus membaca PC word dan TID untuk mengetahui tata letak sebenarnya, bukan mengasumsikan EPC 96 bit.

Skenario ke depan. Kemungkinan terbesar dalam beberapa tahun ke depan adalah: (1) bahasa yang lebih aman (Rust, C dengan analisis statis lebih ketat) makin banyak dipakai di firmware baru; (2) toolchain makin banyak memberi peringatan untuk pola berbahaya (*cast* alignment, alamat anggota packed); (3) inti MCU makin canggih, sehingga pengetahuan tentang cache, barrier, dan alignment yang dulu "opsional" menjadi wajib.

Kronologi

Tahun	Peristiwa
1951–1952	Grace Hopper mengembangkan sistem A-0 untuk UNIVAC I[2]; Alick Glennie mengembangkan Autocode di Inggris[1]
1954	Tim IBM pimpinan John Backus memulai proyek FORTRAN[4]
1957	Compiler FORTRAN I dikirim ke pengguna IBM 704 (April)[4]
1969–1973	Bahasa C dikembangkan di Bell Labs bersama Unix; periode paling kreatif 1972[5]
Pertengahan 1980-an	Komite ANSI X3J11 menstandardisasi C[5]
Akhir 1980-an	GCC dirilis sebagai bagian Proyek GNU
1999	Standar C99 memperkenalkan `<stdint.h>` (`uint32_t` dan kawan-kawan)
2000	Proyek LLVM dimulai di University of Illinois[6]
2003	LLVM 1.0 dirilis[6]
2004–2005	Standar EPC UHF Gen2 diratifikasi; kemudian diadopsi sebagai ISO/IEC 18000-6C (kini 18000-63)
2007	LLRP diratifikasi oleh EPCglobal
2011	C11 memperkenalkan `_Alignas` dan `_Alignof`[14]
2012	Arm menerbitkan AN321 tentang barrier di Cortex-M[32]
2019	Linux mulai menambahkan deteksi *split lock* x86[24]
2021	GS1 menerbitkan LLRP Rilis 2.0[54]
2022	Impinj Indy R2000 memasuki proses akhir masa produksi (September)[50]
2023	C23 menjadikan `alignas` / `alignof` kata kunci; RFC Linux untuk penanganan akses tak-aligned di RISC-V[25]
2026	Patch QEMU untuk perilaku Zicclsm yang benar diajukan[26]

CATATAN KRONOLOGI

Tahun ratifikasi Gen2 (sekitar akhir 2004) dan rilis pertama GCC (1987) ditulis berdasarkan pengetahuan umum industri dan tidak dicek silang secara khusus dalam riset ini. Tanggal "compiler pertama" diperdebatkan, seperti dijelaskan di bab sejarah.

Glosarium

ABI (*Application Binary Interface*): kesepakatan tingkat biner antara kode yang dikompilasi terpisah: ukuran dan alignment tipe, tata letak struct, register untuk argumen, aturan stack. Untuk Arm 32-bit disebut AAPCS.

Alignment: syarat bahwa alamat sebuah objek harus kelipatan angka tertentu (1, 2, 4, 8, ...). *Natural alignment* = alignment sama dengan ukuran tipe.

AST (*Abstract Syntax Tree*): pohon yang mewakili struktur program setelah di-*parse*.

Backscatter: cara tag pasif "menjawab" dengan memantulkan sebagian gelombang pembaca sambil mengubah impedansi antenanya.

Barrier: pagar yang mencegah penyusunan ulang akses memori, baik oleh compiler (*compiler barrier*) maupun oleh prosesor (`DMB` , `DSB` , `ISB`).

Bit-field: anggota struct C selebar beberapa bit; tata letaknya *implementation-defined*.

Cache line: satuan terkecil data yang dipindahkan antara cache dan memori (32 byte di Cortex-M7, 64 byte di x86 modern).

DMA (*Direct Memory Access*): pengendali yang memindahkan data antar-memori/periferal tanpa CPU.

Endianness: urutan byte dalam bilangan multi-byte di memori; *little-endian* (byte terkecil di alamat terkecil) atau *big-endian*.

EPC (*Electronic Product Code*): pengenal barang yang disimpan di bank EPC tag; strukturnya diatur GS1 Tag Data Standard (misalnya SGTIN-96).

HardFault: pengecualian umum di Cortex-M untuk kesalahan berat, termasuk akses tak-aligned di M0/M0+.

IR (*Intermediate Representation*): bentuk program di tengah compiler, tidak terikat bahasa sumber maupun chip target.

ISA (*Instruction Set Architecture*): kontrak antara perangkat lunak dan prosesor: instruksi, register, model memori.

Linker script: file yang memberi tahu linker peta memori chip dan di mana setiap section diletakkan.

LLRP: protokol standar GS1 untuk komunikasi antara pembaca RFID dan aplikasi, di atas TCP.

MMIO (*Memory-Mapped I/O*): register periferal yang diakses seperti alamat memori biasa.

`packed` : atribut compiler yang menghapus padding di dalam struct, menurunkan alignment menjadi 1.

Padding: byte kosong yang disisipkan compiler agar setiap field struct aligned.

PC word (*Protocol Control*): word 16-bit di bank EPC yang antara lain menyatakan panjang EPC (dalam word).

Relokasi: proses linker menambal alamat sebenarnya ke dalam instruksi yang masih berisi "lubang".

RN16: bilangan acak 16-bit yang dikirim tag sebagai jawaban pertama dalam proses inventaris Gen2.

Section (`.text`, `.data`, `.bss`, `.rodata`): bagian-bagian file objek yang berisi kode, data berinisialisasi, data nol, dan konstanta.

SSA (*Static Single Assignment*): bentuk IR di mana setiap variabel ditetapkan tepat satu kali.

Strict aliasing: aturan C bahwa objek hanya boleh diakses lewat tipe yang kompatibel (atau tipe karakter).

Sintesis logika: "compiler untuk hardware" yang menerjemahkan Verilog/VHDL menjadi jaringan gerbang logika.

TID (*Tag Identifier*): bank memori tag berisi identitas chip dari pabrik.

Undefined behavior (UB): perilaku yang tidak dijamin standar; compiler boleh menganggapnya tidak pernah terjadi.

`volatile` : kualifikasi tipe C yang memaksa compiler melakukan setiap akses ke objek itu persis seperti tertulis.

Kesimpulan

Perjalanan dari baris kode ke chip bukanlah satu terjemahan, melainkan **rangkaian kontrak**. Standar bahasa berjanji tentang arti program terhadap mesin abstrak. Compiler berjanji menjaga perilaku yang *bisa diamati*, dan memakai setiap celah lain untuk mempercepat kode. ABI berjanji tentang ukuran, alignment, dan konvensi pemanggilan. ISA berjanji tentang apa yang dilakukan setiap instruksi. Bus dan periferal berjanji bahwa tulisan ke alamat tertentu berarti sebuah aksi fisik. Bug paling sulit di firmware hampir selalu muncul ketika seseorang mengandalkan janji yang **tidak pernah diberikan**: menganggap compiler akan membaca register berulang kali tanpa `volatile`, menganggap prosesor bisa membaca `uint32_t` dari alamat ganjil, atau menganggap DMA dan cache melihat memori yang sama.

Memory alignment adalah contoh terbaik dari prinsip ini. Ia lahir dari fakta fisik (bus dan cache bekerja per word dan per line), dirumuskan menjadi aturan ABI, ditegakkan oleh compiler lewat padding, dan dilanggar oleh programmer lewat *cast* pointer dan struct `packed`. Akibatnya berkisar dari tidak terasa (x86), sedikit lebih lambat (Cortex-M4), sampai *crash* (Cortex-M0+) dan data rusak (DMA dengan cache di Cortex-M7). Solusinya hampir selalu sama: **katakan kepada compiler apa yang benar-benar Anda ketahui**, misalnya dengan `memcpy` atau perakitan byte untuk data tak-aligned, `alignas` untuk buffer yang butuh alignment besar, `volatile` untuk register, barrier untuk urutan, lalu biarkan compiler memilih instruksi terbaik.

Dunia RFID menunjukkan bahwa pola ini berulang di setiap skala. Di dalam tag, protokol diwujudkan sebagai logika yang "dikompilasi" dari HDL, dengan memori yang dialamatkan per word 16-bit. Di dalam chip pembaca, ada MCU dan firmware sendiri yang hanya terlihat sebagai antarmuka SPI. Di MCU host, firmware Anda berbicara dengan chip itu seperti CPU berbicara dengan register. Dan di jaringan, seluruh pembaca menjadi sebuah "periferal" bagi server, dengan format byte (LLRP, big-endian) yang lagi-lagi harus dipisahkan dari tata letak struct di memori. Siapa pun yang memahami satu lapisan dengan baik akan mengenali pola yang sama di lapisan lain.

Pelajaran praktis terpenting: **bedakan selalu antara "berjalan" dan "benar"**. Kode yang berjalan di `-O0`, di laptop, atau di satu jenis MCU belum tentu benar menurut kontraknya. Kode yang benar menurut kontrak akan tetap berjalan ketika compiler diperbarui, optimisasi dinaikkan, atau chip diganti, dan di industri yang umur produknya bisa sepuluh tahun lebih, itulah satu-satunya jenis kode yang layak ditulis.

Bacaan Lanjutan

- **Randal E. Bryant & David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*** (edisi ke-3, 2015). Buku terbaik untuk memahami hubungan C, assembly, linker, memori, dan cache dari sudut pandang programmer. Bab tentang linking adalah sumber banyak bahan kuliah yang dirujuk di sini.
- **Joseph Yiu, *The Definitive Guide to Arm Cortex-M3 and Cortex-M4 Processors*** (edisi ke-3, 2013) dan buku sejenisnya untuk Cortex-M0/M0+ dan M23/M33. Rujukan paling lengkap tentang pipeline, bus, interupsi, dan perilaku akses tak-aligned di Cortex-M.
- **Arm, *Application Note 321: Cortex-M Programming Guide to Memory Barrier Instructions*** (2012)[32]. Singkat, resmi, dan menjelaskan kapan `DMB` / `DSB` / `ISB` benar-benar diperlukan.
- **Arm, *Procedure Call Standard for the Arm Architecture* (AAPCS)** di repositori `ARM-software/abi-aa` [17]. Sumber primer untuk aturan alignment dan konvensi pemanggilan.
- **Keith Cooper & Linda Torczon, *Engineering a Compiler*** (edisi ke-3, 2022). Pengantar desain compiler modern yang lebih praktis daripada "Dragon Book".
- **Ulrich Drepper, *What Every Programmer Should Know About Memory*** (2007). Penjelasan mendalam tentang cache, alignment, dan kinerja memori; berorientasi PC/server tetapi prinsipnya universal.
- **Blog John Regehr** (blog.regehr.org)[1012]. Tulisan-tulisan tajam tentang `volatile`, *undefined behavior*, dan aliasing dari peneliti compiler.
- **GS1, *EPC UHF Gen2 Air Interface Protocol*** dan ***Tag Data Standard***[4457]. Sumber primer untuk memori tag, perintah, dan pengkodean EPC.
- **Klaus Finkenzeller, *RFID Handbook*** (edisi ke-3, 2010). Rujukan klasik tentang fisika, pengkodean, dan protokol RFID di semua pita frekuensi.
- **Compiler Explorer (godbolt.org)**. Bukan buku, tetapi cara tercepat untuk benar-benar *melihat* apa yang dilakukan compiler terhadap kode Anda di berbagai target.

Catatan Kaki

1. CACM Blog, *Did Grace Hopper Create the First Compiler?*, 2022. https://cacm.acm.org/blogs/blog-cacm/268001-did-grace-hopper-create-the-first-compiler/fulltext
2. Centre for Computing History, *Grace Hopper completes the A-0 Compiler*. https://www.computinghistory.org.uk/det/5487/Grace-Hopper-completes-the-A-0-Compiler/
3. IBM, *John Backus* (IBM History). https://www.ibm.com/history/john-backus
4. Wikipedia, *Fortran* (diakses November 2026). https://en.wikipedia.org/wiki/Fortran
5. Dennis M. Ritchie, *The Development of the C Language*, HOPL-II, 1993. https://www.nokia.com/bell-labs/about/dennis-m-ritchie/chist.html
6. Wikipedia, *LLVM* (diakses November 2026). https://en.wikipedia.org/wiki/LLVM
7. Chris Lattner & Vikram Adve, *The LLVM Compiler Framework and Infrastructure*, GCC Summit 2003. https://www-new.llvm.org/pubs/2003-05-01-GCCSummit2003.pdf
8. Technische Universität München, *Code Generation for Data Processing* (buku kuliah), 2024/25. https://db.cit.tum.de/teaching/ws2425/codegen/book.pdf
9. GNU Autoconf Manual, *Volatile Objects*. https://www.gnu.org/s/hello/manual/autoconf/Volatile-Objects.html
10. John Regehr, *Nine ways to break your systems code using volatile*, 2010. https://blog.regehr.org/?p=28
11. OSDev Wiki, *Memory mapped registers in C/C++*. https://wiki.osdev.org/Memory_mapped_registers_in_C/C++
12. John Regehr, *Type Punning, Strict Aliasing, and Optimization*, 2013. https://blog.regehr.org/archives/959
13. ACCU Overload 160, *What is the Strict Aliasing Rule and Why Do We Care?*, 2020. https://accu.org/journals/overload/28/160/anonymous
14. cppreference, *_Alignas (since C11), alignas (since C23)*. https://cpp-ref.vnoi.info/c/language/alignas.html
15. WG14, *N1507: C and C++ Alignment Compatibility*. https://open-std.org/jtc1/sc22/wg14/www/docs/n1507.htm
16. GCC Manual, *Common Type Attributes* (GCC 8.5). https://gcc.gnu.org/onlinedocs/gcc-8.5.0/gcc/Common-Type-Attributes.html
17. Arm, *ABI for the Arm Architecture Advisory Note – SP must be 8-byte aligned on entry to AAPCS-conforming functions*. https://github.com/ARM-software/abi-aa/blob/2a70c42d62e9c3eb5887fa50b71257f20daca6f9/advnote132/advnote132.rst
18. GCC mailing list, *Proposed AAPCS update – parameter passing types with modified alignments*, Juli 2015. https://gcc.gnu.org/pipermail/gcc/2015-July/218022.html
19. ST Community, *Questions about unaligned memory accesses and hard faults*. https://community.st.com/stm32cubeide-mcus-28/questions-about-unaligned-memory-accesses-and-hard-faults-165243
20. Infineon Developer Community, *XMC1400: How to turn off Unaligned Trap*. https://community.infineon.com/t5/XMC/XMC1400-How-to-turn-off-Unaligned-Trap/td-p/300877
21. Launchpad GCC ARM Embedded, *Access to struct on unaligned address with -Os option on Cortex-M0 produces hard fault*. https://answers.launchpad.net/gcc-arm-embedded/+question/239149
22. Agner Fog, diskusi di blog *Agner's CPU blog*. https://www.agner.org/optimize/blog/read.php?i=627&v=t
23. *Rethinking Misalignment to Raise the Bar for Heap Pointer Corruption*, arXiv:1807.01023, 2018. https://arxiv.org/pdf/1807.01023
24. LWN.net, artikel tentang deteksi *split lock*, 2019. https://lwn.net/SubscriberLink/790464/96ddd1213598bb1b/

25. LKML, *[RFC PATCH 0/6] Add support to handle misaligned accesses in S-mode*, 2023. https://lkml.rescloud.iu.edu/2306.3/00153.html
26. Frank Chang (SiFive), *[PATCH v8] target/riscv: Support the true Zicclsm extension*, QEMU, Agustus 2026. https://patchew.org/QEMU/20260810055618.1175500-1-frank.chang@sifive.com/
27. University of Wisconsin–Madison ECE 353, *ARM Cortex-M Memory Map*. https://ece353.engr.wisc.edu/firmware-basics/arm-cortex-m-memory-map
28. Microchip Online Docs, *Behavior of Memory Accesses* (Cortex-M). https://onlinedocs.microchip.com/oxy/GUID-199548F4-607C-436B-80C7-E4F280C1CAD2-en-US-1/GUID-AF2E0CCB-0A02-4BAD-B45B-EB093CF928B2.html
29. Arm Community Blog, *A tour of the Cortex-M3 core*. https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/a-tour-of-the-cortex-m3-core
30. STMicroelectronics, *Arm Cortex-M3 in a nutshell*. https://www.st.com/content/st_com/en/arm-32-bit-microcontrollers/arm-cortex-m3.html
31. Raymond Chen, *The Old New Thing* (seri tentang Thumb-2), Microsoft, Juni 2021. https://devblogs.microsoft.com/oldnewthing/20210601-00/?p=105267
32. Arm, *Application Note 321: Cortex-M Programming Guide to Memory Barrier Instructions* (DAI 0321A), 2012. https://documentation-service.arm.com/static/5efefb97dbdee951c1cd5aaf
33. Arm Community Forum, *DMB/DSB/ISB on Cortex-M3/M4/M7 single core parts*. https://community.arm.com/developer/ip-products/processors/f/cortex-m-forum/10389/dmb-dsb-isb-on-cortex-m3-m4-m7-single-core-parts/33102
34. ST Community, *Maintaining CPU data cache coherence for DMA buffers*. https://community.st.com/stm32-mcus-products-25/maintaining-cpu-data-cache-coherence-for-dma-buffers-11338
35. Ac6, *Cache coherency and DMA on ARM: why your buffers get corrupted*. https://www.ac6-formation.com/en/articles/cache-coherency-dma-arm
36. MikroElektronika, *mikroSDK Reference Manual: CMSIS Core Cache Functions*. https://searchdoc.mikroe.com/group__CMSIS__Core__CacheFunctions.html
37. Rust Embedded, *cortex-m-rt 0.7.6: link.x.in*. https://docs.rs/crate/cortex-m-rt/0.7.6/source/link.x.in
38. Ferrous Systems, *Rust Training: Booting a Cortex-M Microcontroller*. https://rust-training.ferrous-systems.com/latest/book/booting-cortex-m
39. Princeton University COS 217, *Linkers* (slide kuliah), 2003. https://www.cs.princeton.edu/courses/archive/spr03/cs217/lectures/Linker.pdf
40. University of Rochester CSC 254, *Linking* (catatan kuliah), 2020. https://www.cs.rochester.edu/courses/254/fall2020/notes/15b-linking.pdf
41. Colin Walls, *Device registers in C*, Siemens Embedded Software Blog. https://blogs.sw.siemens.com/embedded-software/?p=1600
42. Atlas RFID Store, *Types of Memory in Gen 2 UHF RFID Tags*. https://www.atlasrfidstore.com/rfid-insider/types-of-memory-in-gen-2-uhf-rfid-tags
43. RAIN RFID Alliance, *RAIN RFID Illustrated Glossary*, April 2023. https://therainalliance.org/wp-content/uploads/2023/04/RAIN_RFID_CIN_Illustrated_Glossary_04.2023.pdf
44. GS1, *EPC UHF Gen2 Air Interface Protocol* (standar). https://www.gs1.org/standards/rfid/uhf-air-interface-protocol
45. NXP Semiconductors, *SL3S1206 UCODE 9 Data Sheet*. https://nxp.com/docs/en/data-sheet/SL3S1206.pdf
46. NXP Semiconductors, *UCODE 9xm*. https://www.nxp.com/products/UCODE-9xm
47. US Patent 8,477,015, *System and method for using an input data signal as a clock signal in a RFID tag state machine*. https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/8477015
48. US Patent 9,000,891, *RFID readers, methods and computer program products for adjusting a query command slot-counter parameter Q*. https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/9000891

49. Impinj, *Impinj E700 Series Reader Chips*. https://www.impinj.com/products/reader-chips/impinj-e700-series-reader-chips
50. Impinj, *Indy RS2000 Product Brief* (dengan catatan status akhir masa produksi). https://support.impinj.com/hc/article_attachments/360016175660/impinj-indy-rs2000-product-brief-en-us.pdf
51. RFID Journal, *New Reader Chips to Offer Higher Performance, Flexibility*. https://www.rfidjournal.com/?p=190865
52. STMicroelectronics, *ST25R3916B*. https://www.st.com/en/nfc/st25r3916b.html
53. ST Community, *STSW-ST25R-LIB using SPI while in IRQ context*. https://community.st.com/t5/st25-nfc-rfid-tags-and-readers/stsw-st25r-lib-using-spi-while-in-irq-context/m-p/729301
54. GS1, *Low Level Reader Protocol (LLRP)*, Release 2.0, 2021. https://ref.gs1.org/standards/llrp
55. Wireshark Wiki, *Low-Level Reader Protocol (LLRP)*; lihat juga implementasi `edgexfoundry/device-rfid-llrp-go`. https://wiki.wireshark.org/LLRP
56. GS1 Support, *What is the EPC header for?* https://support.gs1.org/support/solutions/articles/43000734098-what-is-the-epc-header-for-
57. GS1, *EPC Tag Data Standard*. https://ref.gs1.org/standards/tds/
58. TSL (Technology Solutions UK), *LF, HF, dan UHF RFID*. https://www.tsl.com/?p=598
59. STMicroelectronics, *ES0649 Errata sheet: ST25R3916B/ST25R3917B/ST25R3919B/ST25R3920B device limitations*. https://www.st.com.cn/resource/en/errata_sheet/es0649-st25r3916b-st25r3917b-st25r3919b-and-st25r3920b-device-limitations-stmicroelectronics.pdf
60. Arm, *ARMv7-M Architecture Reference Manual* (DDI 0403), bagian tentang dukungan akses tak-aligned dan register CCR. https://developer.arm.com/documentation/ddi0403/latest/
61. crossgcc mailing list, *Memory mapped registers – WORD Access*, November 2001. https://sourceware.org/pipermail/crossgcc/2001-November/007396.html
62. pagetable.com, *Readable and Maintainable Bitfields in C*. https://www.pagetable.com/?p=250
63. Intel Community, *SSE and AVX behavior with aligned/unaligned instructions*. https://community.intel.com/t5/Intel-ISA-Extensions/SSE-and-AVX-behavior-with-aligned-unaligned-instructions/td-p/1170000